



北方民族大学

本科毕业论文（设计）

题目：基于 Vulkan 的实时光线追踪混合渲染器的设计与实现

学院名称：计算机科学与工程学院

专 业：软件工程

学 生 姓 名：向晨

学 号：20221889

指导教师姓名：韩强

企业指导老师：

论文提交时间：

北方民族大学毕业论文（设计）诚信承诺书

学生姓名	向晨	年级	2022 级
所学专业	软件工程	学号	20221889
所在学院	计算机科学与工程学院		
学生承诺 本人慎重承诺和声明： 我承诺在毕业论文（设计）过程中严格遵守学校有关规定，在指导教师的安排与指导下独立完成所规定的毕业论文（设计）工作，决不弄虚作假，不请别人代做毕业论文（设计）或抄袭别人的成果。所撰写的毕业论文或毕业设计是在指导老师的指导下自主完成，文中所有引文或引用数据、图表均注解并说明来源，本人愿意为由此引起的后果承担责任。 学生（签名）： 2024 年 05 月 日			

教务处制

基于 Vulkan 的实时光线追踪混合渲染器的设计与实现

摘要

传统的纯光栅化管线难以自然地呈现物理级的光照与软阴影，而全场景的纯光线追踪受硬件限制，无法满足交互式应用的时效性。本文基于 Vulkan 1.3 显式图形接口，设计和开发了一套渲染引擎架构，并集成实时渲染管线的主流算法。

在底层架构层面，引擎摒弃了繁琐的手动状态机同步，引入了一套声明式的 RenderGraph 调度系统。该模块利用有向无环图自动分析渲染通道间的读写依赖，全盘接管了复杂的显存屏障推导与多线程资源分配。这一设计规避了底层驱动中极易引发的内存读写冲突，也榨取了硬件的并行吞吐潜力。

在核心渲染流转上，系统确立了“光栅化提取几何、硬件光追解算光影”的混合路线。前端利用光栅化构建屏幕空间几何缓冲，后端则调用硬件加速单元计算真实的物理阴影与反射。为了缓解实时光追在极低采样率下产生的蒙特卡洛方差，本项目在光线生成阶段优化了采样策略，引导探测射线根据光源能量权重进行重要性采样，从源头上遏制了原始画面高频噪点的爆发。

针对残留的低频光影瑕疵，管线末端集成了时空方差引导滤波与时域抗锯齿联合重建机制。系统通过提取运动矢量复用前一帧的历史画面，并配合保留几何边缘的空间滤波，以极其微小的算力开销将破碎的噪点还原为真实光影。

关键词：Vulkan；三维渲染引擎；实时光线追踪；计算机图形学

Design and Implementation of a Real-time Ray Tracing Hybrid Renderer Based on Vulkan

Abstract

Traditional pure rasterization pipelines struggle to naturally present physical-level lighting and soft shadows, while full-scene pure ray tracing is constrained by hardware limitations, failing to meet the real-time requirements of interactive applications. Based on the Vulkan 1.3 explicit graphics API, this thesis designs and develops a rendering engine architecture and integrates mainstream algorithms for real-time rendering pipelines.

At the underlying architecture level, the engine abandons cumbersome manual state machine synchronization and introduces a declarative RenderGraph scheduling system. Utilizing a Directed Acyclic Graph to automatically analyze read-write dependencies between render passes, this module completely takes over the complex derivation of memory barriers and multi-threaded resource allocation. This design circumvents memory read-write conflicts that are highly prone to occur in underlying drivers, and maximizes the parallel throughput potential of the hardware.

In terms of the core rendering flow, the system establishes a hybrid route of "rasterization for geometry extraction and hardware ray tracing for lighting calculation." The front-end utilizes rasterization to rapidly construct screen-space geometry buffers (G-Buffers), while the back-end invokes hardware acceleration units to calculate realistic physical shadows and reflections. To alleviate the Monte Carlo variance generated by real-time ray tracing under extremely low sampling rates, this project optimizes the sampling strategy during the ray generation stage, guiding probe rays to perform importance sampling based on light source energy weights, thereby curbing the outbreak of high-frequency noise in the raw image from the source.

Targeting the residual low-frequency lighting artifacts, the end of the pipeline integrates a joint reconstruction mechanism combining Spatiotemporal Variance-Guided Filtering (SVGF) and Temporal Anti-Aliasing (TAA). By extracting motion vectors to reuse historical frames and

coordinating with edge-preserving spatial filtering, the system restores fragmented noise into realistic lighting with minimal computational overhead.

Key words: Vulkan; 3D Rendering Engine; Real-Time Ray Tracing; Computer Graphics

目录

第一章 绪论	2
1.1 研究背景及意义	2
1.2 研究现状	3
1.3 本文研究内容	4
1.4 论文总体结构	4
第二章 实时混合渲染器相关技术	6
2.1 实时混合渲染技术	6
2.1.1 光栅化	6
2.1.2 光线追踪	7
2.2 基于物理的渲染	8
2.2.1 微面元理论	8
2.2.2 能量守恒	8
2.2.3 基于物理的 BRDF	9
2.3 实时渲染管线	10
2.3.1 前向渲染管线	10
2.3.2 延迟渲染管线	11
2.4 图形 API	11
2.4.1 OpenGL 及其他图形 API	11
2.4.2 Vulkan	12
2.5 本章小结	14
第三章 需求分析	15
3.1 可行性分析	15
3.2 系统业务流程分析	15
3.3 系统功能性需求分析	16
3.3.1 用户功能需求	16

3.4 系统非功能性需求分析	18
3.4.1 性能和响应时间:	18
3.4.2 安全性与稳定性	18
3.4.3 可靠性	19
3.4.4 易用性	19
3.4.5 可维护性与可扩展性	19
3.5 系统核心类设计	20
3.5.1 全局管理与框架类设计	20
3.5.2 资源管理与场景组织类设计	21
3.5.3 渲染调度引擎核心类设计	22
3.6 本章小结	22
第四章 概要设计	23
4.1 引擎的架构设计	23
4.1.1 Walnut	23
4.1.2 RenderGraph	24
4.1.3 本引擎架构设计	25
4.2 功能模块划分	26
4.2.1 图形硬件后端模块	26
4.2.2 资源管理模块	26
4.2.3 场景管理模块	27
4.2.4 渲染调度模块	27
4.2.5 混合渲染管线模块	27
4.2.6 交互调试模块	27
4.3 混合渲染管线流程设计	27
4.3.1 几何特征采集	28
4.3.2 物理光影计算	29
4.3.3 时空信号重建	29
4.3.4 光照合成与后期处理	29

4.4 本章小结	29
第五章 详细设计	30
5.1 图形硬件后端模块	30
5.1.1 基于 volk 的动态函数指针加载机制	30
5.1.2 物理特性按需注入机制	31
5.1.3 交换链管理与异常恢复	33
5.2 资源管理模块	33
5.2.1 无绑定资源池	34
5.2.2 动态加载：基于引用计数的异步流转机制	34
5.2.3 光源预处理：蒙特卡洛积分 CDF 硬件同步	35
5.3 场景管理模块	37
5.3.1 光追加速结构动态构建机制	37
5.3.2 基于八叉树的视锥体剔除策略	38
5.3.3 时域抖动注入与历史矩阵追踪系统	38
5.4 渲染调度模块	39
5.4.1 任务编排与资源依赖的声明式建模	40
5.4.2 自动同步屏障推导与状态转换机制	40
5.4.3 异构计算任务的执行上下文封装	42
5.5 混合渲染管线模块	42
5.5.1 基于光栅化的几何特征提取	42
5.5.2 单样本光线追踪探测	43
5.5.3 SVGF 降噪实现	45
5.5.4 光照合成与后期处理管线	45
5.6 交互调试模块	47
5.6.1 应用生命周期与事件驱动机制	47
5.6.2 非侵入式参数反控与闭环调优	48
5.6.3 图形界面后端与性能可视化	48
5.7 本章小结	49

第六章 系统实现与测试	50
6.1 系统运行环境	50
6.2 系统功能实现	50
6.2.1 交互式调试界面展示	50
6.2.2 G-Buffer 几何特征可视化.....	51
6.2.3 光线追踪与降噪算法对比	52
6.2.4 后期处理与最终画面输出	52
6.3 系统测试	52
6.4 本章小结	53
第七章 总结与展望	54
7.1 全文工作总结	54
7.2 研究不足与未来展望	54
参考文献	56
致谢	58

第一章 绪论

1.1 研究背景及意义

实时渲染是专注于制作和分析实时图像的计算机图形学的子领域，其旨在极短的时间内完成从三维几何数据到二维图像的计算与生成。在典型的实时交互应用中，系统需要持续捕获用户的输入，更新虚拟环境状态，并以较高的刷新率输出连续的图像序列。通常，帧率（FPS）^①大于 30 的渲染被称为实时渲染，10FPS 左右的我们称为可交互渲染，再慢的就是离线渲染。实时渲染技术目前广泛应用在强交互的 3D 游戏和带 3D 的应用中，例如，游戏，VR/AR，虚拟仿真等，有巨大的应用前景和商业价值。

在技术路线上，实时渲染技术主要分为光栅化（Rasterization）与光线追踪（Ray Tracing）两种类型^[1]。光栅化技术基于多边形投影原理，其算法流程高度契合 GPU 的单指令多数据流（SIMD）并行计算架构，能够以极高的效率处理大规模几何数据。然而，光栅化本质上是一种基于局部信息的计算模型，缺乏对场景全局几何拓扑的直接感知能力。因此传统光栅化管线只能通过屏幕局部像素推算、或是提前烘焙存储光照信息的方式，间接实现全局光照、软阴影和物理反射等效果。这类近似处理与预存储方案在处理复杂遮挡关系和动态场景时，容易产生图像走样与物理光影失真等缺陷。光线追踪技术则是通过在三维空间中模拟光线的物理传播、散射与折射路径，能够更准确地求解渲染方程，从而生成具备物理真实感的光影效果。但由于光线在场景中弹射路径具有较高的随机性，难以充分利用 GPU 的并行算力，在很长一段时间都主要应用在离线渲染领域。

随着 GPU 硬件算力的提升以及专用硬件光线追踪加速单元的引入，实时光线追踪技术逐渐具备了工程可行性。为平衡渲染质量与计算开销，混合渲染架构（Hybrid Rendering Architecture）成为当前的主流发展方向。该架构将光栅化在初级可见性（Primary Visibility）计算上的高效率，与光线追踪在次级光线求解上的准确性相结合，有效弥补了单一管线的不足，是目前图形学界研究的主流方向。

^① 通常会使用每秒显示的帧数（frames per second, FPS）或者赫兹（Hertz, Hz）来衡量图像显示的速率。

1.2 研究现状

图形渲染管线及底层硬件经历了长期的演进过程。20 世纪 90 年代末，图形硬件主要采用固定渲染管线（Fixed-Function Pipeline），开发者仅能通过配置预设状态来控制渲染输出。2001 年起，可编程着色器（Programmable Shaders）的引入使开发者能够自定义顶点变换与像素着色逻辑。至 2006 年，统一着色器架构（Unified Shader Architecture）的提出，打破了不同着色阶段计算资源的物理隔离，为现代通用 GPU 计算奠定了基础。2018 年是实时渲染的分水岭，Microsoft 发布了 DXR（DirectX Raytracing）框架，并且 NVIDIA 在同年推出了基于 Turing 架构的 RTX 平台^[2]，让实时光线追踪成为可能。同年，EA 旗下 SEED 团队发布的 Demo “Pica Pica”^[3, 4]，是业界首个基于混合渲染管线案例。

在底层图形 API 方面，传统 API 由于设计于单核 CPU 时代，采用了高度封装的全局状态机模型。在这种架构下，内存管理与线程同步操作由图形驱动程序隐式处理，导致在处理复杂场景时产生较高的 CPU 驱动开销，限制了多核处理器的并发能力。随着对性能的追求，显式图形 API 如 Vulkan 与 DirectX 12 诞生。此类 API 将显存的分配与释放、多线程命令缓冲区的录制以及管线同步屏障（Pipeline Barriers）的控制权显式移交至应用层，显著降低了驱动开销，提升了硬件算力的利用率。

渲染器是游戏引擎中核心的子系统之一，它负责图形绘制，接收渲染命令和资源数据（顶点、纹理、Shader 等），并调用 OpenGL/D3D/Vulkan 等图形 API，将游戏世界中的数据（模型、纹理、光照、摄像机等）转换为最终呈现在屏幕上的图像。最早可追溯至 id Software 开发的游戏《德军总部 3D》《毁灭战士》^[5, 6]，其核心开发者 John Carmack 将可复用的底层代码进行剥离，创造了游戏引擎这一概念。DOOM 引擎（id Tech 1）是业界首个支持商业授权的游戏引擎；随后推出的 Quake 引擎（id Tech 2）则是真正意义上的 3D 引擎，推动了显卡革命。当前商业游戏引擎市场主要由 Unreal 与 Unity 主导，开源领域亦涌现出 Orge, Godot 等引擎；同时，专业离线渲染引擎（如 V-Ray、OctaneRender）也给许多引擎和渲染开发者提供了优秀的学习资料，共通促进了游戏引擎，渲染领域的发展。

在渲染引擎架构层面，随着渲染管线复杂度的增加，传统的面向对象绘制指令流逐渐难以维护。现代工业级渲染引擎广泛引入了基于数据驱动的渲染图架构 RenderGraph^[7]。

该架构通过构建有向无环图（DAG），在编译期自动推导各渲染通道（Render Pass）之间的数据依赖关系，从而实现显存的动态复用与同步屏障的自动插入，是目前处理复杂混合渲染管线状态调度的标准方案。

另一方面，实时光线追踪对时效性的要求极其严格。由于当前的硬件性能限制，目前主流方式是在实际渲染时采用非常低的采样率。通过重采样技术，获取额外采样信息，为了弥补画质，之后在后处理阶段对噪声进行滤波。在降噪算法中，Christoph Schied 等人提出了 SVGF 时域滤波器^[8]，时间上复用前一帧的历史数据，也使用空间滤波。该降噪算法的执行效率较好和画面效果也还不错。

1.3 本文研究内容

本项目是使用 Vulkan 设计并实现了一个三维实时混合渲染引擎。主要研究内容如下：

1. 使用现在 Vulkan 特性构建引擎渲染层。探索资源和场景管理的工程构建和算法实现。
2. 参考现有的引擎架构，分析并设计出本项目的引擎架构，实现输入、事件等渲染引擎的基本功能。
3. 抽象 Vulkan 的底层 API，设计并实现 RenderGraph 调度系统，寻找高效管理渲染管线的方法。
4. 设计基于延迟渲染的混合管线。首先使用光栅化在第一阶段提取数据到几何缓冲，之后利用 Vulkan 硬件光线追踪扩展处理光照，然后使用 SVGF 降噪，最后经过后处理生成图像。

1.4 论文总体结构

本文共七章，具体内容安排如下：

第一章 绪论。交代实时混合渲染的课题背景。梳理图形硬件、图形 API 以及渲染算法的发展现状，并总结本文的开发任务。

第二章 理论基础。介绍分析光栅化、光线追踪、PBR 物理模型和图形 API 等实时渲染相关理论。

第三章 需求分析。从可行性、业务流程、功能与非功能指标等分析渲染引擎需求。

第四章 概要设计。通过分析现有技术引出本渲染引擎和渲染管线的设计。

第五章 详细设计。详细阐述第四章重点说明的模块。

第六章 实现与测试。展示项目实现效果，通过具体的测试用例分析引擎实现的完整度。

第七章 总结与展望。本文的结束部分，对本文的主要工作进行总结，分析当前引擎架构存在的局限性，展望未来的优化改进策略。

第二章 实时混合渲染器相关技术

本章将对构建实时渲染器所涉及的相关技术进行介绍，包括渲染技术，基于物理的渲染，混合渲染管线和图形 API 介绍。

2.1 实时混合渲染技术

渲染是指通过软件算法将包含几何、视点、纹理和照明等信息的三维（或二维）模型最终转换为二维图像的技术，其过程可类比为物理相机的成像原理，即将三维空间信息投影至二维平面之上。可以看出渲染的数据源主要包含模型，相机和光源位置。实时渲染技术主要有下面两种方法。

2.1.1 光栅化

光栅化是现代实时渲染中最经典、应用最广泛的技术之一。光栅化可以理解为将 3D 空间中的三角形投射到 2D 屏幕^②上的一个过程。其外层循环遍历场景中的几何图元，内层循环寻找覆盖该图元的屏幕像素。

在光栅化管线中，第一步是完成坐标空间的转换。通过 MVP 变换把三维世界中的顶点投影到二维的屏幕中。完成映射后，三角形面会通过硬件的固定管线离散化为屏幕上的点。这个过程通常采用边方程或重心坐标算法进行内外测试，逐一判断像素中心是否落在了当前三角形的覆盖范围内。

在处理复杂场景的遮挡关系时，系统主要依靠深度缓冲（Z-Buffer）机制。在流程中会在显存中分配了两块与屏幕分辨率大小相同的二维数组，分别存储当前屏幕所有点的颜色和深度。如果有多个三角形在屏幕上的投影是同一个屏幕像素时，GPU 会进行深度测试：对比新深度值 Z_{new} 与缓冲区里记录的历史深度值 Z_{old} ，保留靠近屏幕的顶点，同时更新记录新的点的相关数据。

光栅化的优势是契合 GPU 的并行计算架构，能很好处理初级可见性的问题。光栅化算法在着色阶段（Shading Stage）本质上是一种严格的局部计算模型。在现代图形硬件的片段着色器（Fragment Shader）中，大规模并行处理的各个计算单元仅能访问当前片段的局部属性（如插值后的法线、纹理坐标、材质参数）以及全局一致变量（Uniforms，

^② 在图形学中我们可以把它抽象为一个二维数组，其中二维数组中的每个元素是像素（pixel）

如直接光源方向），缺乏对场景全局几何拓扑信息的感知能力。由于无法直接探测场景中的其他几何体，光栅化架构下的阴影、环境遮蔽与反射通常依赖于启发式近似（Heuristics）或预处理技术（如 Shadow Maps、Environment Maps），这些技术在处理动态场景与复杂遮挡关系时往往存在走样（Aliasing）或精度缺失的问题。这对物理光照模型做了简化，导致光影效果相对简单，无法完全模拟真实世界中的复杂光照现象。

2.1.2 光线追踪

与光栅化的投射思路不同，采用基于采样的策略。其外层循环遍历屏幕上的二维像素，内层循环则在三维场景中寻找与该像素视线相交的几何图元。

光线追踪采取了一种逆向模拟光路物理传播的思路，从摄像机发射光线，与物体表面相交后，根据材质的粗糙度、折射率继续反弹，直至与光源相交或达到最大迭代深度 [9, 10]。

在算法表达上，光线追踪通过逆向模拟光路传播，从观察点向空间发射射线。射线的数学参数方程定义为：

$$P(t) = O + t \cdot D \quad (2-1)$$

其中 O 为射线原点， D 为标准化的方向向量， t 为光线传播的标量距离。算法的核心算力被消耗在求解射线 $P(t)$ 与场景中所有三角形平面的代数交点上。

为了避免 $O(N)$ 级别的暴力求交导致的性能崩溃，现代光线追踪算法需依赖空间加速结构（Acceleration Structure）。通常采用层次包围盒（Bounding Volume Hierarchy, BVH）或 KD 树，将场景几何体在三维空间中进行层次化切分，使得单根光线求交的时间复杂度从线性的 $O(N)$ 骤降至对数级别的 $O(\log N)$ 。当射线与场景中的几何体的空间加速结构相交时，可以逆推出该点的入射光线方向，根据交点处的材质属性，生成反射光线、折射光线以及阴影光线。通过逐层递归计算，最终将各层的光照结果累积起来，得到像素的颜色。

相较于光栅化，光线追踪能够更好地还原物理光照效果，在全局光照、镜面反射、透明折射、软阴影等细节表现上更具优势，画面真实感更强，能够呈现出接近现实世界的视觉效果。但由于采样的随机性，无法适应 GPU 的 SIMD 并行架构，需要大量光线迭代求交运算，整体计算成本极高，传统纯光线追踪渲染难以满足高帧率实时交互的需求，通常用于离线渲染、影视特效等对实时性要求较低的场景。直到近几年，NVIDIA、

AMD 等硬件厂商在 GPU 中加入了 RT Core（硬件光追核心），才让实时光线追踪成为可能。

2.2 基于物理的渲染

在早期的图形学中，我们广泛使用的是 Phong 或 Blinn-Phong 光照模型。这些模型被称为经验模型，是对光照的一种模拟，通过实践总结出简化的方法，简化了真实的光照计算。这种传统模型的参数有较强的随意性和技巧性(tricks)，在不同的光照条件、引擎下是不通用的。为了让材质的工作流统一，PBR 诞生了。早期的 PBR 优势并不明显，自迪士尼在 SIGGRAPH 2012 上提出了著名的“迪士尼原则的 BRDF(Disney Principled BRDF)”之后，由于其良好的应用性和便捷的工作流，推动了 PBR 渲染革命^[1]。PBR 并不是一个单一的公式，而是一套基于真实世界光学物理法则的渲染规范。其核心理论有三个：微面元理论、能量守恒以及基于物理的 BRDF。

2.2.1 微面元理论

该理论认为，任何在宏观视觉上平滑的表面，在微观尺度下都是由无数个朝向各异的微小平面（微面元）构成的。如图 2.1 所示，材质的粗糙度直接决定了这些微面元法线的分布规律：表面越光滑，微面元的朝向越高度一致，反射光线越平行集中，从而形成明亮且锐利的镜面高光；表面越粗糙，微面元的朝向越杂乱无章，反射光线被散射到各个方向，从而形成宽广而黯淡的高光^[12]。

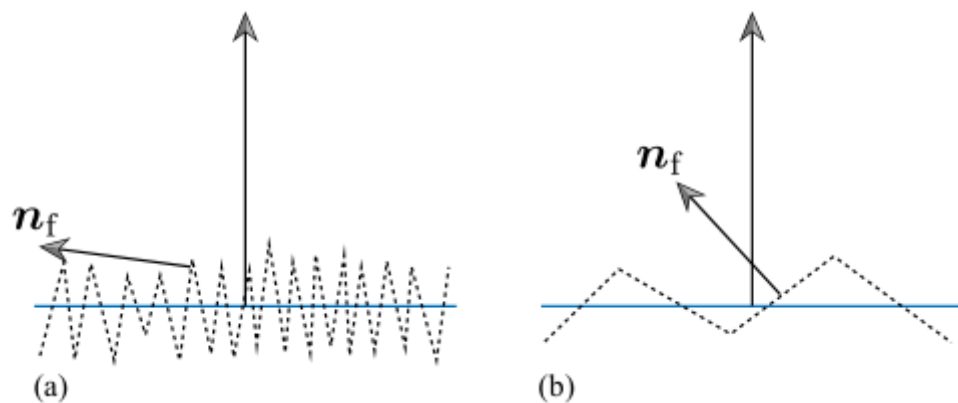


图 2.1 微表面模型原理图。(a) 粗糙表面；(b) 光滑表面

2.2.2 能量守恒

基于物理学的能量守恒要求：出射光的总能量不能超过入射光的总能量。一束光打在介质表面后，必然分为两部分：立刻从表面弹射的反射光（Specular），以及穿透表面

进入内部被吸收或散射出的漫反射光（Diffuse）。表面直接反射的光越强，能够进入内部参与漫反射的光就越少，这样来维持物理上的真实感^[13]。

2.2.3 基于物理的 BRDF

双向反射分布函数模型（BRDF），当给定入射角度后，BRDF 可以给出所有出射方向上的反射和散射光线的相对分布情况。现代 PBR 通常采用 Cook-Torrance BRDF 模型^[14]，它将光照计算科学地拆分为漫反射项与镜面反射项。通项公式如下：

$$f_r(l, v) = k_d f_{diffuse} + k_s f_{specular} \quad (2-2)$$

式中， k_d 和 k_s 分别是漫反射和镜面反射的能量权重系数， $f_{lambert}$ 是 Lambert 漫反射模型，其计算表达式为：

$$f_{lambert} = \frac{c}{\pi} \quad (2-3)$$

c 表示表面的基础反照率（Albedo），即表面颜色。除以 π 是为了保证漫反射在半球空间积分下的能量守恒。

$f_{cook-torrance}$ 则为 Cook-Torrance 镜面反射模型，专注于模拟微面元在物理层面的高光贡献如下：

$$f_{cook-torrance} = \frac{D(h)F(v, h)G(l, v, h)}{4(n \cdot l)(n \cdot v)} \quad (2-4)$$

式中， n 表示物体表面法线， v 表示观察方向， l 表示光源入射方向， h 表示视线与光线的半程向量。 DFG 这三个函数分别用来描述微表面的粗糙度分布、菲涅尔效应和几何遮挡效应，常用的 DFG 函数有多个选择。

法线分布函数从统计学的角度近似地表示了与半程向量 h 取向一致的微表面的比率。本文采用 Trowbridge-Reitz GGX 模型来进行估算^[15]，见式 2-5

$$D(\vec{n}, \vec{h}, \alpha) = \frac{\alpha^2}{\pi \left((\vec{n} \cdot \vec{h})^2 (\alpha^2 - 1) + 1 \right)^2} \quad (2-5)$$

GGX 模型相较于传统的 Blinn-Phong 分布，具备更平滑的拖尾（Tail）过渡特征，能够渲染出更具层次感的物理高光边界。公式中的 α 为材质的表面粗糙度系数。

几何函数描述了表面的自遮挡程度。微观平面的凹凸不平会让光线被平面自身遮挡。几何函数描述了自遮蔽后的能量强度，可以拆分为两部分，光线被遮挡的部分(shadowing)和视线被遮挡的部分(masking)。一个完整的 G 函数由两部分组成：

$$G(n, v, l, k) = G_{sub}(n, v, k) \cdot G_{sub}(n, l, k) \quad (2-6)$$

一个可以选择的 G 的公式为 schlick-GGX 近似，其形如：

$$G_{sub}(n, v, k) = \frac{n \cdot v}{(n \cdot w)(1 - k) + k} \quad (2-7)$$

其中 k 是一个和粗糙度 roughness 有关的常数。系数 k 通常可以用以下公式：

$$k = \frac{(\alpha + 1)^2}{8} \quad (2-8)$$

菲涅尔方程 (F 项)：描述现实世界中极为普遍的现象——当观察视线越趋近于平行表面（掠射角）时，表面的反射率越强。

$$F(h, v, F_0) = F_0 + (1 - F_0)(1 - (h \cdot v))^5 \quad (2-9)$$

其中 F_0 是材质表面在垂直视角下的基础反射率。

2.3 实时渲染管线

在了解了光栅化和光线追踪的基本原理后，我们需要知道这些理论是如何在现代游戏引擎中被组织并执行的。这就引出了渲染管线（Rendering Pipeline）的概念。

渲染管线本质上是一套流水线化的数据处理流程：CPU 提交的 3D 顶点数据（Vertices），经过几何变换、光栅化、片段着色，最终转化为屏幕上的 2D 像素（Pixels）。随着人们对渲染效果的要求不断提高，渲染管线也经历了从前向渲染到延迟渲染的变革，两者的主要区别根据光照计算时机的不同。

2.3.1 前向渲染管线

前向渲染是最基础、最直观的渲染方式。目前市面上几乎所有的移动端轻量级游戏，以及原生的 OpenGL/Vulkan 教学案例，默认采用的都是前向管线。前向渲染的核心逻辑是：渲染器会遍历场景中的每个物体，对于物体上的每个可见像素，计算场景中所有光源对该像素的光照贡献，最后输出到颜色缓冲区。其可见性测试（深度测试）与着色计算（Shading）是同一阶段耦合进行的。但当多个物体在屏幕上重叠时，一个像素可能会被多个重叠的三角形覆盖。而这些被遮挡的物体碎片仍然可能执行复杂的着色计算。这种多余的计算在场景中有多个光源时会产生极大浪费，适合光源较少的情景。前向渲染的优点是它天然支持半透明物体（Translucency）渲染，并且能够兼容硬件级的多重采样抗锯齿（MSAA）。

2.3.2 延迟渲染管线

延迟渲染管线是现代的高效渲染管线技术，适用于光源较多、场景复杂的应用。将可见性测试与着色计算完全解耦，主要可以分为两个阶段。

首先是几何处理阶段：渲染器不进行任何光照计算，只遍历场景中的物体，将场景中每个像素的几何信息和材质属性（如法线、反照率、粗糙度、深度）提取出来，分别渲染到几张屏幕大小的纹理中，这组纹理被统称为 **G-Buffer**。由于有深度测试（**Z-Test**），**G-Buffer** 中最终留下的一定是离摄像机最近的、不会被遮挡的像素数据。

然后是光照处理阶段：在这一步，我们不考虑 3D 几何体，只在屏幕上绘制一个全屏的 2D 矩形。片段着色器会读取 **G-Buffer** 中每个像素的属性，然后结合场景中的光源进行光照计算。现在无论场景有多复杂，重叠了多少层物体，每个屏幕像素永远只计算一次光照，避免了光照的 **Overdraw** 问题。

延迟渲染减少了光照计算的开销，但是缺点也很明显。**G-Buffer** 需要在极高的分辨率下存储大量的浮点数信息，会占用大量的显存带宽（**Bandwidth**）。由于 **G-Buffer** 中存储的是离散的属性数据，传统的 **MSAA** 无法直接作用于光照结果，需要使用 **TAA**（时域抗锯齿）等后处理抗锯齿方案。并且 **G-Buffer** 每个像素只能存储一层数据，无法记录半透明物体背后的信息。

2.4 图形 API

图形应用程序接口（**Graphics Application Programming Interface**，简称图形 API），是连接应用程序与图形硬件的软件接口，负责将应用程序中的渲染指令（如几何绘制、光照计算、纹理采样等）转换为 GPU 能够识别和执行的指令，是构建实时混合渲染器的重要工程基础。目前业界主流的图形 API 主要有 **OpenGL**、**DirectX**、**Metal** 和 **Vulkan**，各自具有不同的特点和适用场景，需开发者根据渲染器的开发需求和目标平台进行权衡。

2.4.1 OpenGL 及其他图形 API

OpenGL 是图形学领域历史最悠久、应用最广泛的跨平台图形 API 之一。于 1992 年由 **Silicon Graphics, Inc. (SGI)** 开发，在 2006 年转为由非营利组织 **Khronos Group** 管理。**OpenGL** 采用的是基于全局状态机的设计模型。其开放性强、生态成熟，拥有丰富的函数库和工具支持，API 易于使用，降低图形编程的门槛——开发者无需关心内存是如何分配的，也无需管理底层的多线程同步。但也因为大部分底层操作由 **OpenGL** 自己

处理，导致每次绘制调用（Draw Call）前都需要进行繁重的状态合法性校验，有巨大的性能瓶颈。在 2017 年发布 OpenGL4.6 后，Khronos Group 转而全力开发 Vulkan 图形 API，OpenGL 已经停止更新。

DirectX 是由微软公司开发的图形 API，仅支持 Windows 和 Xbox 系列游戏主机平台，是 Windows 平台下游戏开发的主流 API。DirectX 经过多代迭代，目前最新版本为 DirectX 12。DirectX 11 提供了极佳的稳定性和完善的工具链，至今仍有大量游戏采用。DirectX 12 顺应了底层优化的趋势，引入了显式内存管理和多线程指令提交，在高端实时渲染、3A 游戏开发中具有显著优势。

Metal 是苹果公司在 2014 年苹果为 macOS、iOS 等系列产品定制的图形和计算 API，优势在于能最大程度地发挥苹果自研芯片的内存架构优势，提供了低开销（Low-overhead）和直接访问 GPU 的能力。

2.4.2 Vulkan

Vulkan 是 Khronos Group 推出的新一代跨平台图形 API^[16]。与 OpenGL 传统的隐式全局状态机设计不同，Vulkan 采用了显式（Explicit）的架构理念。如图 2.2 所示，Vulkan 提供一个更精简的驱动层，应用层获得更多控制权则能更精细化的控制渲染流程，给用户提供了更加底层的 API 调用，这种设计增加了 Vulkan 的门槛，但换来了巨大的性能提升。另外，Vulkan 官方提供的 VK_KHR_ray_tracing_pipeline 扩展非常完善^[17]。这也使得我们能在统一的 API 框架下，直接调用 RTX 等显卡的硬件光追单元。

在现代图形引擎开发中，Vulkan 提供了以下的现代特性：

指令队列与多线程提交。Vulkan 不允许直接把指令塞给 GPU 硬件。GPU 的算力被划分成了图形（Graphics）、计算（Compute）和传输（Transfer）等不同的物理队列。在实际编写中，开发者需要先开辟命令缓冲（Command Buffers），把画图指令录制进去，最后再统一进行队列提交（Queue Submit）。这种显式设计非常适合多核 CPU 发挥优势。可以在后台线程无锁并行录制指令，提高引擎渲染效率。

同步屏障。Vulkan 驱动把指令之间的依赖管理交给了开发者。为了防止“写后读（RAW）”这种数据冒险导致画面撕裂或崩溃，开发者必须手动插入同步屏障。在编写 RenderGraph 调度器时，我们需要精准设置源阶段（Source Stage）、目标阶段以及对应的访问掩码。只有严格设置屏障，才能保证 G-Buffer 真正写入显存之后，后续的光追

探测阶段再去进行读取。

显式同步原语(Fence 与 Semaphore)。除了管理指令流,Vulkan 还把 CPU 与 GPU 之间的同步交给了开发者。围栏(Fence)主要负责“CPU 等待 GPU”。例如,CPU 提交完当前帧的渲染指令后,使用围栏将自己挂起,直到确认 GPU 彻底画完了这一帧,才去回收显存或重置命令缓冲。而信号量(Semaphore)则专门处理 GPU 内部不同物理队列之间的协同(例如让图形队列等待计算队列的输出结果),这个过程完全在显卡内部闭环执行,不会打断 CPU 的运行节奏。

内存布局转换。GPU 里的图像资源/Images)在不同的使用阶段,需要采用不同的底层内存排列方式来最大化带宽利用率。例如,一张纹理在当做渲染目标写入时,必须是 COLOR_ATTACHMENT_OPTIMAL 布局;等到了着色器里作为贴图被采样时,又必须切成 SHADER_READ_ONLY_OPTIMAL 布局。这种状态切换全靠开发者插入显存屏障来触发,驱动程序收到指令后才会去执行底层 L1/L2 缓存的刷新与重排。

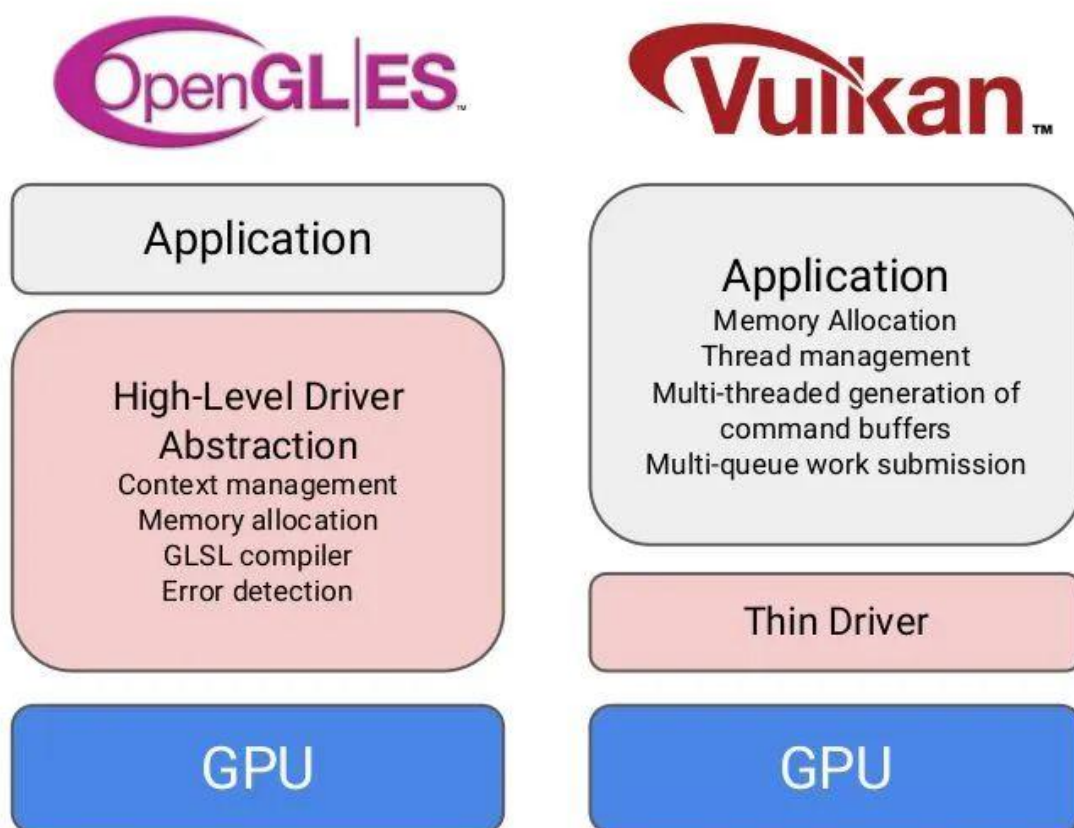


图 2.2 OpenGL Vulkan API 对比图

2.5 本章小结

第二章主要为后续的引擎开发铺垫了理论与工程底座。一开始，我们对比了光栅化和光线追踪的底层原理。接着，为了让渲染出来的材质不失真，我们详细拆解了 PBR 体系。重点梳理了 Cook-Torrance BRDF 模型，涵盖了微面元理论以及 D 、 F 、 G 的公式介绍。在管线架构方面，解释了为什么现代引擎大多放弃了前向渲染，转而使用延迟渲染（G-Buffer）来解耦几何与光照计算。最后，我们剖析了 Vulkan 的显式架构，介绍指令队列与内存屏障的调用机制。

第三章 需求分析

在正式编写底层渲染代码之前，本章主要从软件工程的视角，梳理开发本混合渲染引擎的需求。通过对项目可行性、系统业务运转流程以及性能指标的评估，为后续的架构设计定下具体的设计基准。

3.1 可行性分析

作为一款偏向底层研发的个人毕业设计，项目的可行性主要体现在以下三个维度。

经济与资源可行性：本项目的研发成本低，完全基于开源生态。开发所需的 Vulkan SDK、MSVC 编译器均可免费获取。同时在开源世界的许可证也有 MIT 这种限制极小的，开源项目也分别会选择合适的许可证以扩大受众范围。我在选用和集成第三方库时会考虑到开源世界的声誉和功能的需求，最终选用的第三方库们对我的项目没有多大限制。在硬件平台方面，目前主流的消费级显卡已经原生集成了成熟的 RT Core 光追加速单元，无需额外采购昂贵的工作站即可完成开发测试。

技术实现可行性：随着图形学 API 和 GPU 硬件的发展，Vulkan 1.3 规范与 KHR 光线追踪扩展目前已非常成熟。遇到底层内存同步等疑难 Bug 时，可直接利用 RenderDoc 这种专业的图形抓帧工具进行逐像素分析。庞大的开源社区也为解决管线设计瓶颈提供了充足的技术参考。

操作与验证可行性：为方便后续对画面进行调优，需要为引擎设计一套可视化面板。幸好开源和商用引擎的代码参考实现较为丰富，用户熟悉主流的相机，交互的实现。如果实现主流的，经过市场验证的外观，能很好得降低用户和开发者的门槛。

3.2 系统业务流程分析

当启动这款混合渲染器时，系统内部的数据流转与业务流程主要遵循以下步骤：第一步是环境初始化，程序启动后会立即绑定 Vulkan 上下文，并拉起前端窗口界面；紧接着进入场景解析阶段，引擎会在后台将用户指定的 3D 模型与 HDR 天空盒数据载入显存；随后是最核心的管线分配阶段，底层的 RenderGraph 调度器会根据用户当前选择的模式（例如纯光栅化或光线追踪），自动在后台选择渲染路径，分配所需的 G-Buffer 并串联起各个 Render Pass；在画面持续输出过程中，便进入交互调优阶段，用户可随时通

过菜单栏切换 SVGF 降噪的开关，或调整相机的观察角度；最后，主界面的视口不仅会输出最终的物理光影画面，还允许开发者随时切换查看场景的深度图、法线图中间渲染结果，便于排查画面出错的原因。

3.3 系统功能性需求分析

本渲染器的核心受众为图形开发者，因此在功能设计上需围绕“场景搭建、效果对比和性能监控”三个方向展开。

3.3.1 用户功能需求

系统必须要能自动扫描并解析目录下的 GLTF 模型与环境贴图。在 UI 层级面板中，用户选中某个特定实体后，可以实时修改它的平移、旋转和缩放参数。这里的核心要求是：只要物体的空间位置发生了变化，底层必须要立刻触发并更新硬件加速结构（BLAS/TLAS），否则光追射线的求交计算就会出现错位。

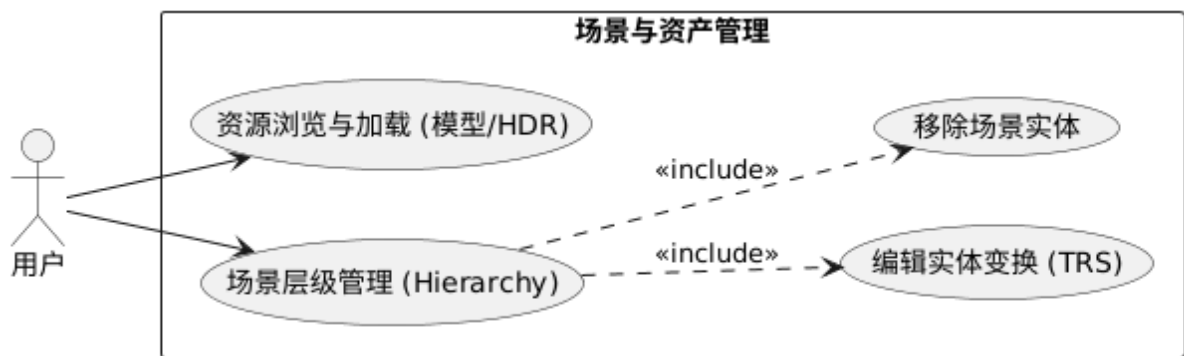


图 3.1 场景与资产管理用例图

为了直观对比画质，面板上需要提供切换渲染路径的按钮。以及需要体现渲染管线的中间视图，方便 debug 测试。在信号重建模块，允许用户单独开启或关闭 SVGF 降噪与 TAA 抗锯齿，用来验证降噪算法的实际效果，也方便开发者修改。此外，Debug 面板需要实时打印出相机的当前世界坐标、每秒帧率（FPS），以及各个 Render Pass 在 GPU 上的具体毫秒开销。

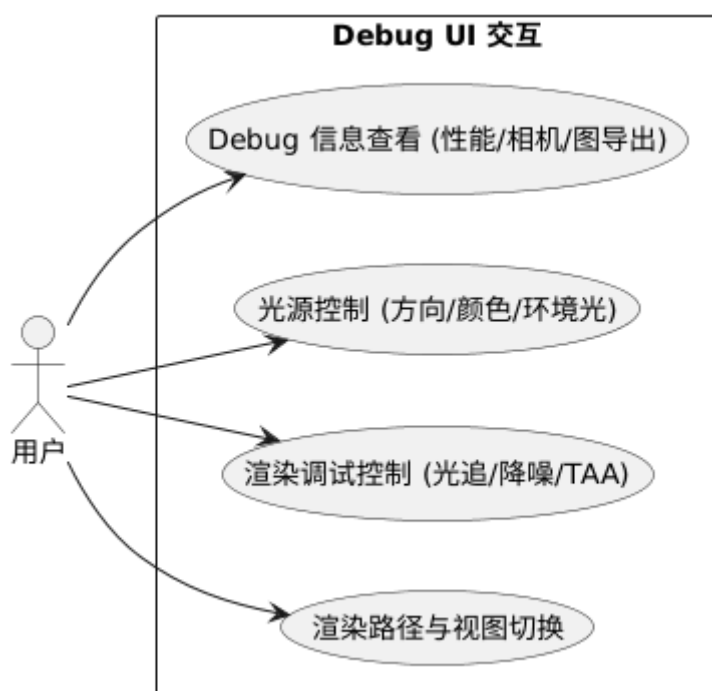


图 3.2 渲染控制与参数配置用例图

为了全方位观察光影细节，需要手写一套类似成熟建模软件的弧球（Arcball）相机逻辑。用户通过按住 Alt 键配合鼠标的左中右键，就能极其丝滑地实现视角的环绕旋转、焦点平移和焦距缩放，做到对场景 360 度无死角的巡检。

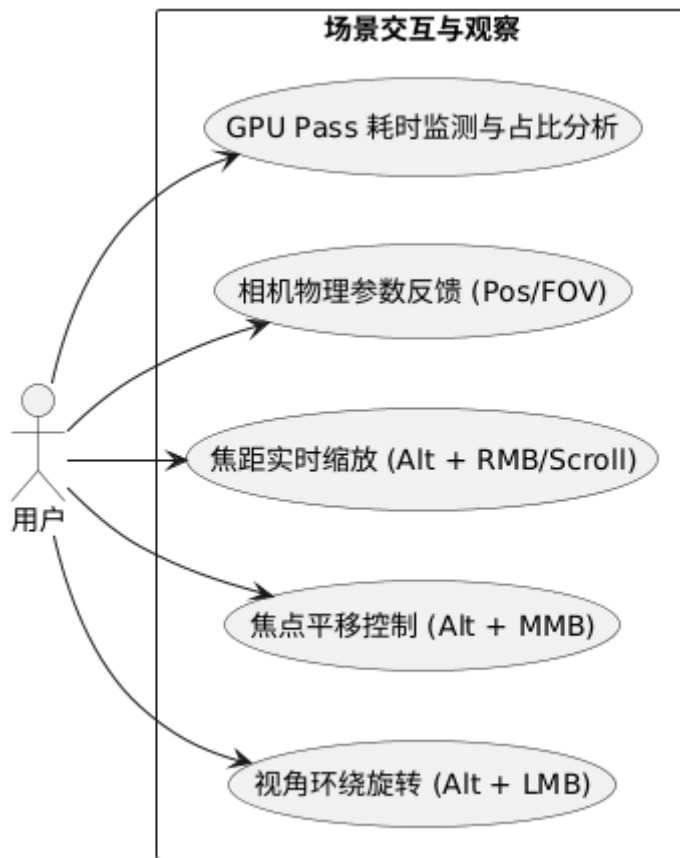


图 3.3 场景交互与观察用例图

3.4 系统非功能性需求分析

非功能需求规定了系统在运行过程中需遵守的品质限制和技术规范。在此基础上，从性能、安全性、可用性、易用性和可维护性等五个方面，分析面向实时混合渲染的非功能性标准。

3.4.1 性能和响应时间：

对于主打实时交互的图形引擎，渲染性能是衡量其可用性的首要指标。在目标测试平台（如配备 RTX 系列独立显卡的设备）上，系统需保证在多光源、高面数复杂场景下的端到端延迟控制在 16 毫秒左右，以维持 60 FPS 的稳定运行。具体到混合渲染管线的内部调度，硬件资源的使用效率需高度优化。

3.4.2 安全性与稳定性

与 OpenGL 不同，Vulkan 将内存分配和多线程同步的控制权完全交给了开发者。因此，本引擎的稳定性需求主要集中在内存管理与状态机同步上。首先是显存资源的安

全边界问题。在混合渲染的数据流转中，G-Buffer 纹理、光追加速结构以及 SVGF 依赖的历史帧缓冲(History Buffers)会频繁地被创建和更新。系统需实现一套基于 VMA^[18] (Vulkan Memory Allocator) 的引用计数管理逻辑，避免出现“GPU 正在读取某张纹理，而 CPU 端已将其销毁”的非法访问现象。其次是管线执行状态的正确性。需要能准确推导出所有的图像布局转换(Image Layout Transitions)和内存访问屏障。若遗漏关键的读写同步屏障，将导致渲染画面出现块状闪烁、贴图撕裂，甚至触发设备丢失(Device Lost)报错，迫使显卡驱动强行终止程序。

3.4.3 可靠性

系统的可靠性主要体现在面对硬件兼容性问题或非法操作时的自我恢复机制。由于光线追踪高度依赖特定的底层硬件扩展，系统在初始化的第一阶段，必须具备严格的硬件特性探测逻辑。如果检测到当前显卡不支持 VK_KHR_ray_tracing_pipeline 扩展，引擎不应直接崩溃退出，而是应当在后台输出详细的特性缺失诊断日志，并安全地将管线降级回纯光栅化渲染分支。此外，在用户进行高频 UI 交互时（例如在运行时动态切换渲染模式），底层的交换链(Swapchain)、帧缓冲容器以及描述符集(Descriptor Sets)会面临大规模的销毁与重构。在管线状态机重建期间，系统必须保证上一帧的历史数据被彻底清理，且显存状态转换过程合法，防止发生内存越界访问。

3.4.4 易用性

系统集成的 Dear ImGui^[19]交互面板应符合数字内容创作(DCC)工具的行业操作习惯。例如，在排查光影瑕疵时，开发者极度依赖对中间渲染数据的观测。系统应当允许用户通过下拉菜单，将主视口画面快速切换为基础反照率、世界空间法线(World Normal)、深度图或者是 SVGF 运行过程中的空间方差图(Variance Map)。这种直观的视觉化调试手段，能够省去借助外部抓帧工具的繁琐步骤。配合平滑的弧球摄像机(Arcball Camera)平移与缩放机制，可有效降低用户观察光追暗部高频细节时的操作成本。

3.4.5 可维护性与可扩展性

引擎的架构设计需为未来的图形学算法演进提供足够清晰的接入规范。这就要求系统在代码层面上严格贯彻“逻辑调度层、资源抽象层与底层驱动层”的三层结构解耦。基于 RenderGraph 声明式的管线构建思路，如果未来需要集成新的图形特性（例如接入改进的 TAA 时间抗锯齿或泛光 Bloom 效果），开发者只需编写对应的 Shader 逻辑，并

在 C++ 层声明该 Pass 的输入输出依赖即可。核心调度中枢能够自动将其编排进现有的混合光照数据流中。新功能的接入不需要对底层的 Vulkan 同步代码进行破坏性的修改。这种高度模块化、接口透明的系统架构，是降低二次开发门槛、提升代码可维护性的核心保障。

3.5 系统核心类设计

本节通过一系列类图展示系统的静态设计结构，涵盖从全局框架到具体渲染调度的协作关系。系统采用高度模块化的 C++ 设计，其核心类协作主要分布在框架管理、资源调度、场景组织及渲染执行四个主要维度。

3.5.1 全局管理与框架类设计

系统以 Application 为单例核心，通过组合模式聚合 Window 和 VulkanContext 确立运行环境。分层框架利用 std::vector<std::shared_ptr<Layer>> 构建逻辑栈，支持 EditorLayer 等功能层级的一致性分发。其中 VulkanContext 对 VulkanInstance 与 VulkanDevice 进行了深度封装，为全系统提供统一的硬件能力抽象。

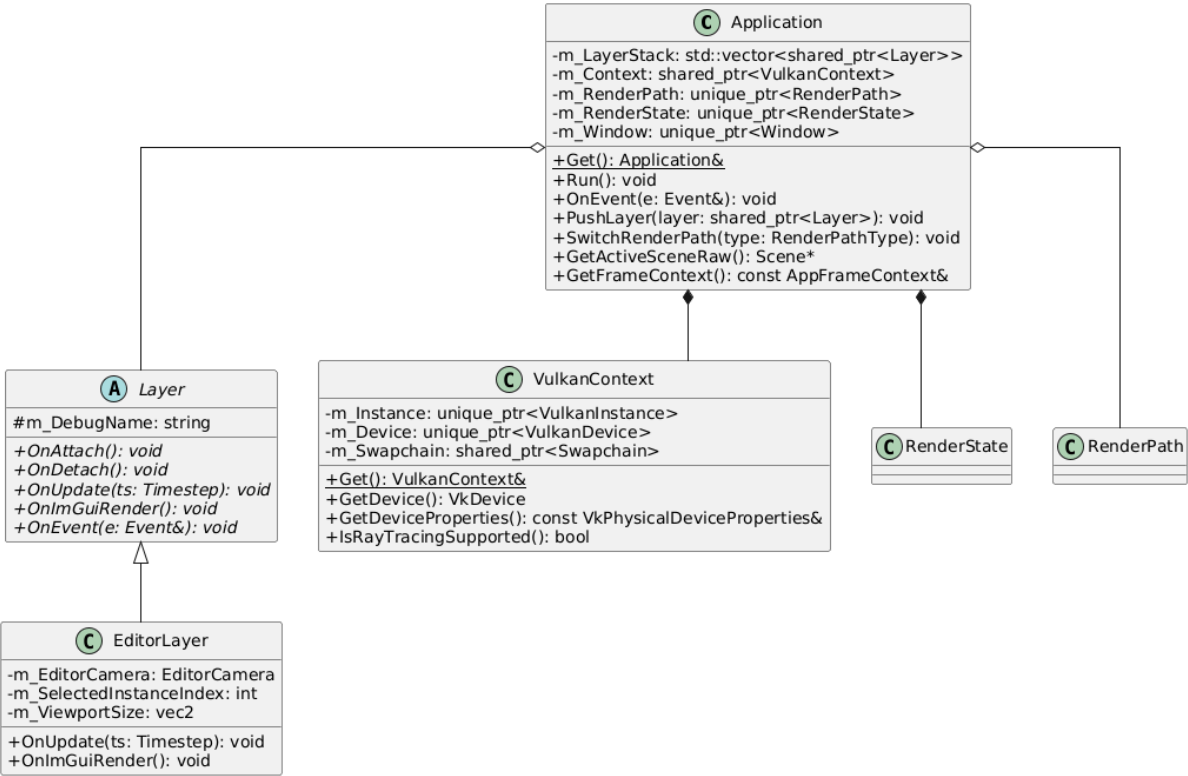


图 3.4 全局管理与分层框架类图

3.5.2 资源管理与场景组织类设计

系统采用组件化(Component-based)的场景组织结构。ResourceManager 负责 Vulkan 资源的 RAII 封装，通过基于线程池(Thread Pool)的多线程后台处理流水线，实现资产加载与逻辑更新的并行化，确保了大规模场景导入时的系统响应效率。

Scene 类通过 Entity 容器组织场景对象，每个实体由 TransformComponent (TRS 变换) 与 MeshComponent (模型引用) 构成。系统通过对这些组件的实时追踪，自动维护并同步硬件加速结构(AS)。

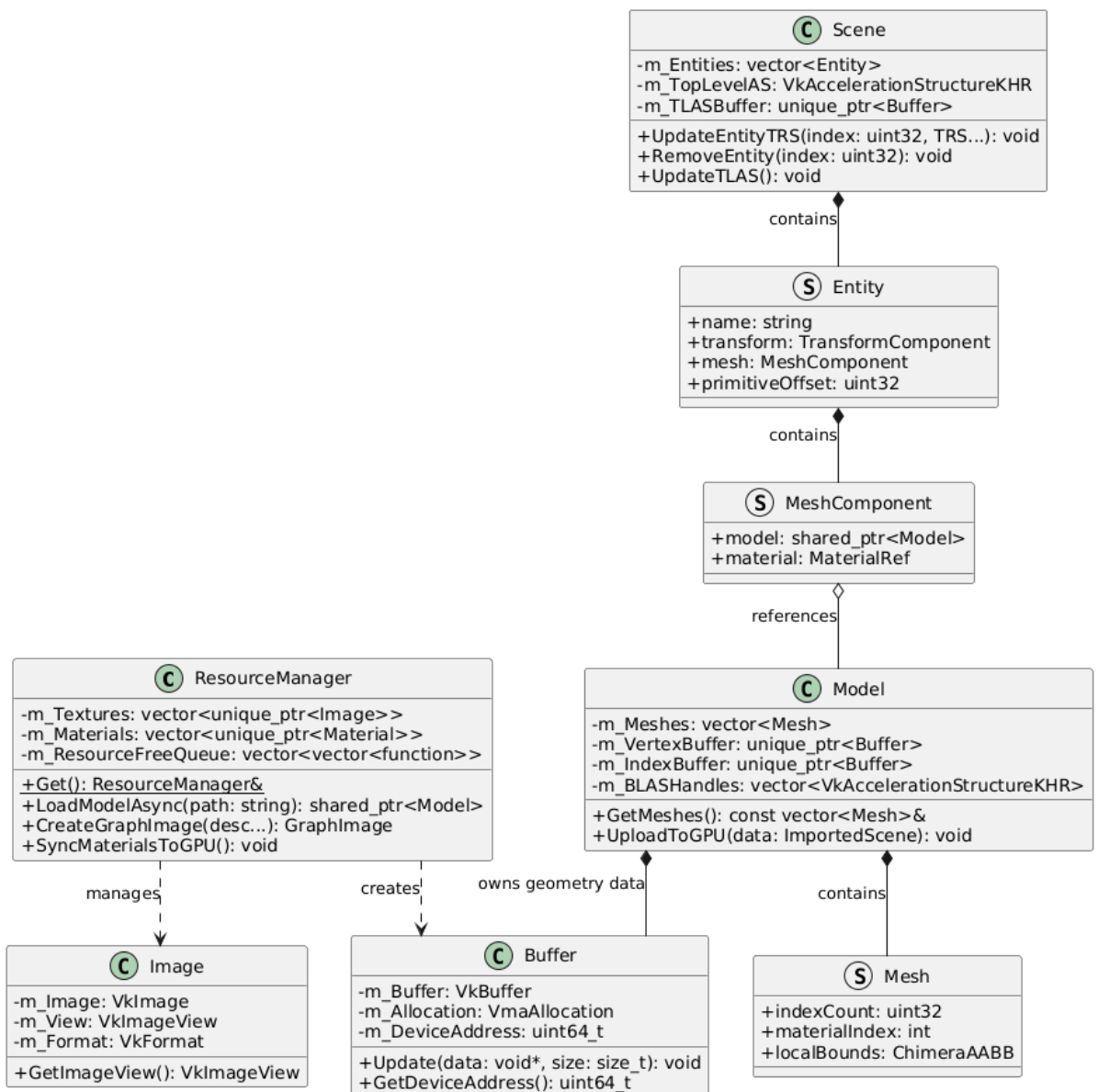


图 3.5 资源管理与场景组织类图

3.5.3 渲染调度引擎核心类设计

渲染调度由声明式的 `RenderGraph` 引擎驱动。系统通过 `PassBuilder` 捕获各 `Render Pass` 对显存资源的读写（`Read/Write`）依赖，从而推导出资源的物理生存区间。在执行阶段，引擎为不同类型的通道分发对应的执行上下文（`ExecutionContext`），屏蔽了底层同步屏障与图像布局转换的复杂性，实现了高层次的架构解耦。

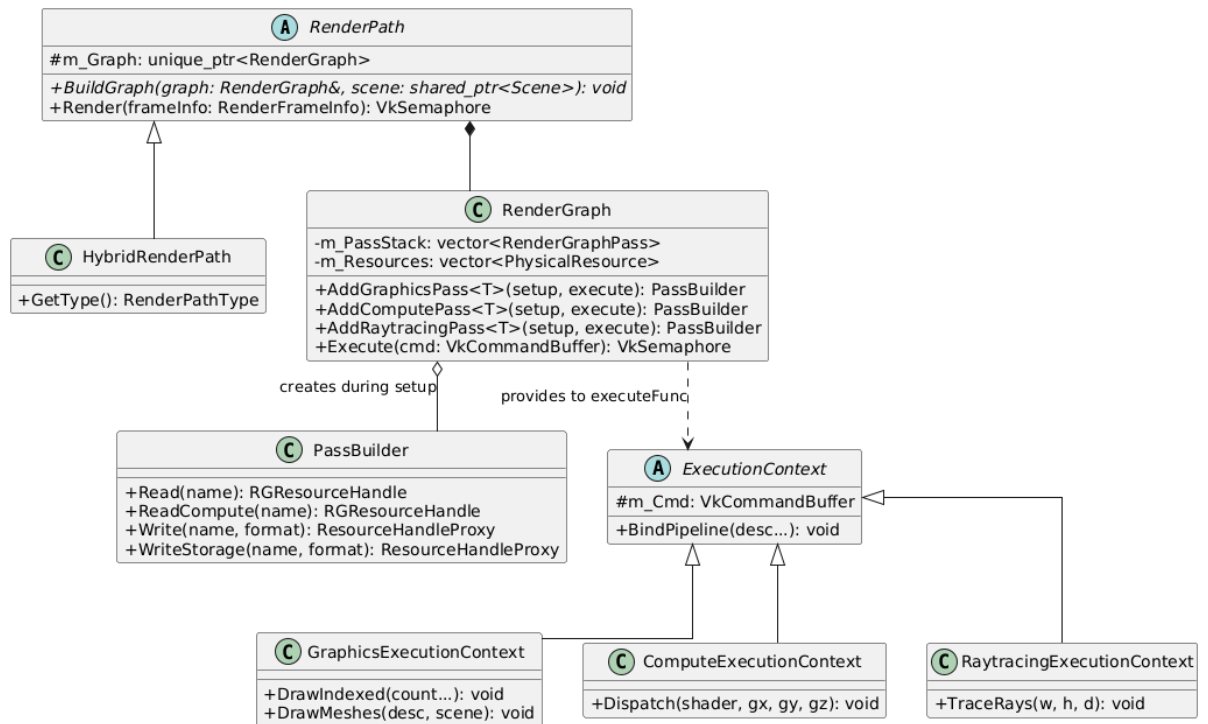


图 3.6 `RenderGraph` 调度引擎类图

3.6 本章小结

这一章主要在正式编码前确立了引擎的需求标准。一开始，我们盘点了当前的软硬件开发环境，确认了不借助商业许可也能搭建起一套前沿的光追架构。接着理清了引擎启动后的核心业务流转流程，并确立了模型加载、管线切换、Arcball 相机漫游等必要的交互功能。随后，在非功能需求方面，给渲染器的帧率底线、底层显存防崩溃机制以及模块解耦定下了死命令。把这些前提条件和类结构梳理清楚后，下一章就可以进入核心混合渲染架构的规划与设计环节了。

第四章 概要设计

本章将从软件工程的视角出发，对混合渲染引擎进行概要设计。从参考引擎和优秀设计出发，借鉴并设计出本引擎的架构与模块。

4.1 引擎的架构设计

随着多年的发展，目前有许多成熟的引擎可供参考。本项目在底层架构上参考了结构清晰的 Walnut 框架，而在核心图形调度上则引入了现代化的 RenderGraph 系统。本节将对这两种核心技术架构进行理论分析，并引出其在本项目中的设计。

4.1.1 Walnut

Walnut^[20]是从游戏引擎 Hazel 中抽象出来的微内核框架，它剔除了游戏引擎中繁重的物理、音频、脚本等模块，仅保留了最核心的渲染循环与窗口管理逻辑。

Walnut 使用了微内核与层叠式架构（Layer Stack）。如图 4.1 所示，整个框架被严格划分为应用外壳、逻辑层栈和平台抽象层。其关键机制在于确立了基于生命周期回调函数（Lifecycle Callbacks，如重载 OnUpdate、OnRender、OnImGuiRender 方法）的渲染管线切入与事件分发系统。框架在每一帧通过自底向上的遍历，依次触发各逻辑层的回调，实现了渲染画面与交互调试 UI 的分离，也让本项目在开发混合渲染管线时，无需关注底层的窗口重建与系统级事件路由细节。

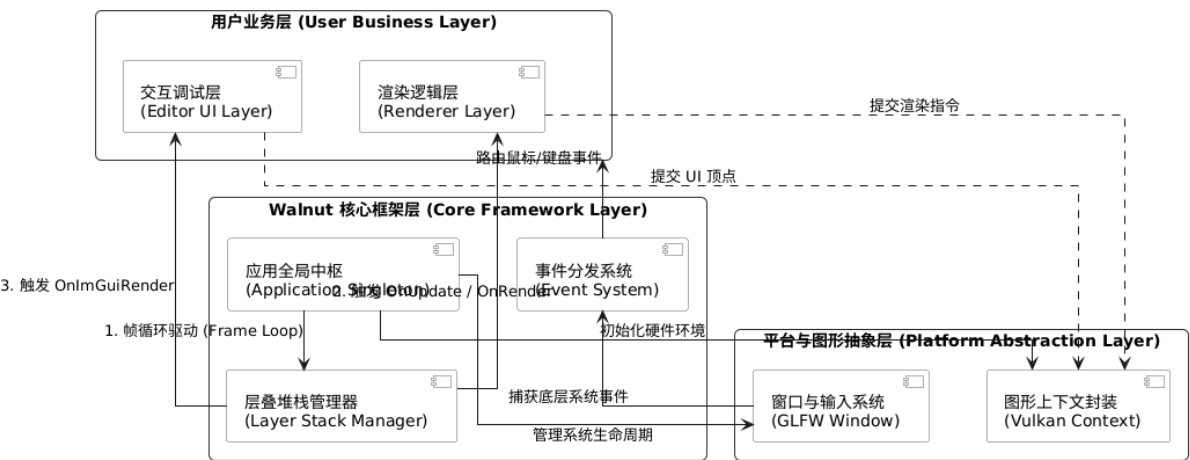


图 4.1 Walnut 框架组件架构图

4.1.2 RenderGraph

现代图形 API 将内存管理、多线程管理等大部分工作交由开发者负责，这赋予了开发者极高的硬件控制自由度，但也增加了 API 使用的复杂度和困难度。例如，仅利用原生 Vulkan 绘制单一基础图元便需构建上千行的前置代码。

软件工程的核心诉求之一即为复杂度管理。现代引擎需实现一个中间层架构，在利用图形 API 性能优势的同时，有效降低其复杂度，并让整体渲染管线更加可扩展、易于调试。此类作为渲染调度中枢的架构即为渲染图（RenderGraph）。

RenderGraph（也叫 FrameGraph）最早由 EA 寒霜引擎（Frostbite Engine）团队在 GDC 2017 大会上首次提出。它主要由 Render Pass 和 Resource 构成：

Render Pass 是一份元数据，定义了一个完整的渲染流程需要用到的所有资源。

Resource 是指 Render Pass 使用的 Render Target、Texture、Shader 等物理资产。

每个 Render Pass 都会显式指定对应的 Input 和 Output 资源。通过这种依赖关系，Render Pass 和 Resource 就形成了一个严密的 DAG 结构。

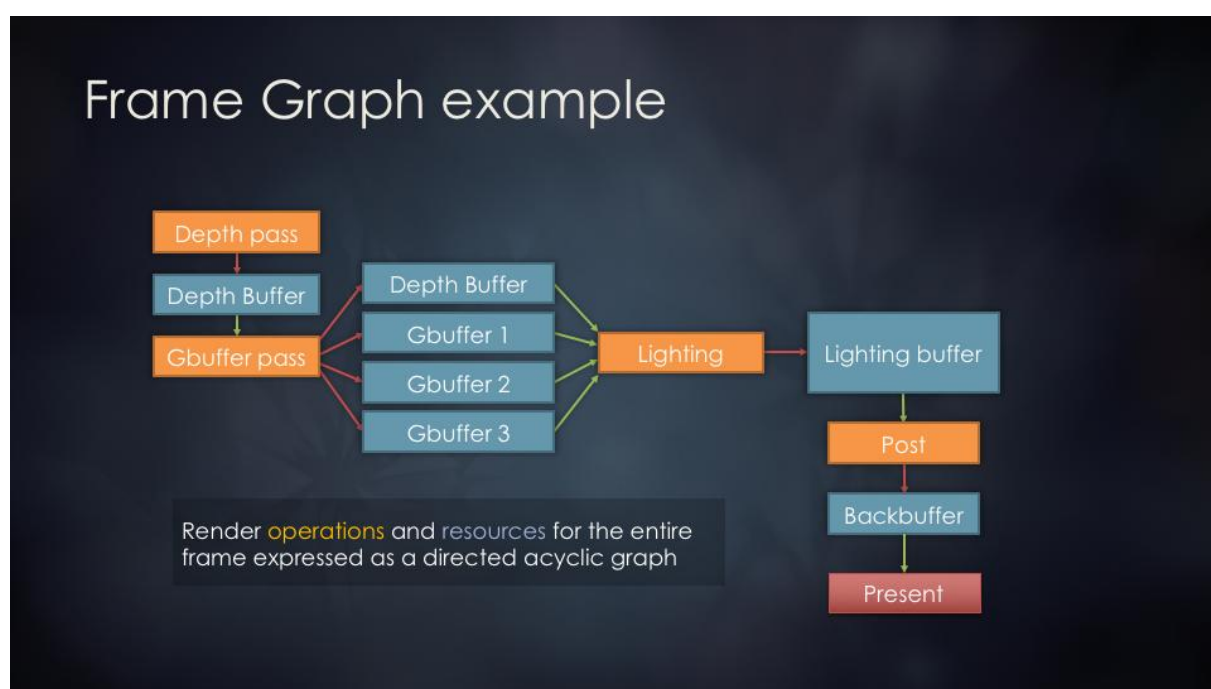


图 4.2 RenderGraph DAG 可视化示例图

RenderGraph 拥有完整一帧的全部信息，通过分析各个 Render Pass 之间依赖关系可以方便地对资源做优化。DAG 也可以很方便的做到可视化，方便对现代复杂渲染管线的调试。可以自动化推导 Pass 执行顺序、自动插入同步屏障，并利用显存复用

（Memory Aliasing）减少整体的内存占用。

4.1.3 本引擎架构设计

本项目借鉴了 Walnut 与现代游戏引擎的通常的架构思路^[21]，并结合 RenderGraph 调度架构，将系统自底向上划分为五个拓扑层级。采用模块化设计能解耦业务逻辑与底层 API，为后续的功能扩展预留了标准化的接口。

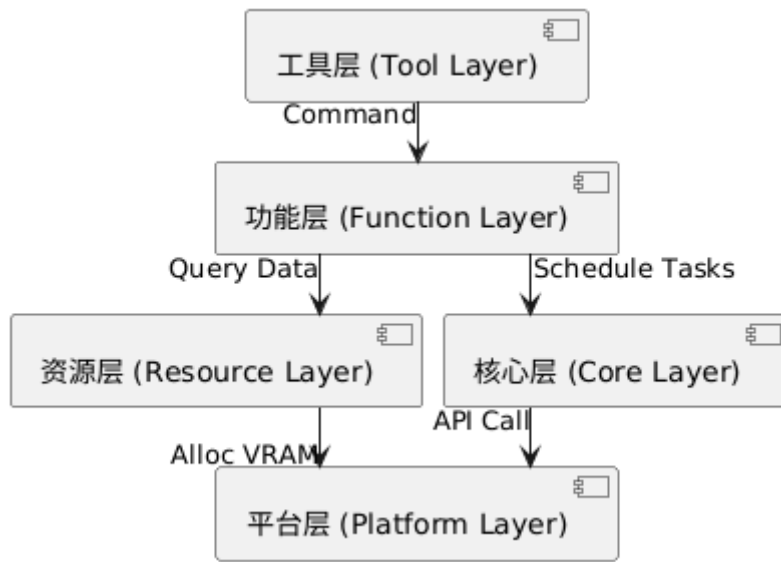


图 4.3 引擎五层分层拓扑架构图

平台层位于架构最底部，充当引擎的硬件抽象层（HAL）。它的核心任务是对接操作系统与 Vulkan 1.3 接口，封装底层的通信细节。由于显式 API 的调用逻辑极其繁琐，系统必须建立统一的通信边界，以防止上层的图形算法被特定的显卡驱动所绑定。这种隔离设计能屏蔽了系统窗口句柄，提升代码的跨平台能力。开发者得以专心编写渲染逻辑，无需再反复处理枯燥的底层初始化工作。

核心层是支撑引擎运转的基础设施，其主要工作是突破底层的性能瓶颈。该层内部集成了任务并行调度器，负责向各个 CPU 核心分发多线程的指令录制任务。由于 Vulkan 的手动显存管理极易引发内存泄漏，核心层设置基于 VMA 的分配策略，专门负责大块显存的切分与回收。此外，考虑到光追算法对算力的严苛要求，核心层需要封装数学运算库，为整个引擎提供统一的代数运算接口。

资源层负责管理场景中所有资产的生命周期。它将磁盘上的 GLTF 模型和 HDR 贴图读取进内存，转化为 GPU 可直接访问的 Buffer 或 Image 对象，且整个加载流程严格

遵守 RAII 原则。针对光线追踪任务，资源层还需要动态维护硬件加速结构。一旦场景中的实体发生位移或形变，资源层会立刻捕捉这些状态变化，并自动触发底层的 BLAS 与 TLAS 更新，从而严格保证射线求交时的物理精度。

功能层是整个引擎的“心脏”，承载了最核心的 RenderGraph 调度系统。它的职责是把抽象的渲染意图，精准翻译成 GPU 的真实执行流。系统会自动分析各个渲染节点（Pass）之间的资源读写依赖，RenderGraph 则据此推导出最优的执行顺序，并智能插入所需的同步屏障与内存布局转换指令。无论是光栅化提取 G-Buffer，还是落地光追反射与 SVGF 降噪算法，所有的核心数据流转都是在这一层完成统筹调度的。

工具层位于架构顶端，直接面向最终开发者。该层集成了 Dear ImGui 交互界面，通过调用功能层开放的接口来实现可视化的参数调优与场景监控。用户可以在面板上随时切换渲染路径，动态调整 SVGF 的时空方差参数，或者精准查看每一个渲染 Pass 的毫秒级耗时。工具层方便开发者画质的精细化对比，也为引擎的性能排查提供了最直观的数据支撑。

4.2 功能模块划分

在五层垂直架构的基础上，本节将系统按业务逻辑选出了渲染相关的功能模块。

4.2.1 图形硬件后端模块

该模块主要承担抹平图形 API 复杂度、提供显式硬件控制的核心职责。其功能涵盖三个维度：一是图形上下文建立，负责逻辑设备初始化及呈现队列管理；二是显存池化管理，利用核心层设施实现基于 VMA 的高效显存亚分配，规避驱动层频繁申请导致的性能波动；三是描述符布局自动推导，为后续光线追踪管线提供底层的描述符资源池（Descriptor Pool）支持。

4.2.2 资源管理模块

该模块专职负责海量三维资产的摄入与显存流转。其核心功能是构建非阻塞式的资产加载流水线，将 glTF 2.0 等标准资产解析为硬件可极速访问的顶点缓冲与图像对象。针对混合渲染需要随机访问全局材质的严苛特性，该模块实现了无绑定（Bindless）材质资源管理系统，将场景中所有的纹理资源统一映射至全局描述符数组中，从底层打破了传统渲染器逐对象绑定（Per-Object Binding）的性能瓶颈。

4.2.3 场景管理模块

该模块负责维护物理世界的空间拓扑一致性，是驱动硬件光线追踪的绝对先决条件。其核心功能分为两部分：一是空间划分与视锥体剔除，利用包围盒（AABB）算法筛选出当前视角的活跃实体，减少冗余绘制开销；二是加速结构动态维护，负责实时追踪场景内实体的运动位移，并动态重构底层几何结构（BLAS）与顶层场景结构（TLAS），确保 GPU 的光线探测单元（RT Core）能够时刻感知绝对精确的物理世界分布。

4.2.4 渲染调度模块

该模块是引擎实现“数据驱动渲染”的思想大脑。它完全围绕 RenderGraph 展开：通过在 CPU 端收集各项渲染意图并构建 DAG，在 GPU 提交前执行拓扑排序，自动化推导出最优的 Pass 执行序列。其核心价值在于消除了手动维护 Vulkan 图像内存屏障（Image Memory Barriers）的沉重工程负担，并实现了生命周期不重叠资源的智能显存复用（Aliasing）。

4.2.5 混合渲染管线模块

该模块是引擎图形算法的集中体现。它将混合渲染流程解构为四个高度解耦的核心子单元：几何特征采集单元（生成 G-Buffer）、光影探测单元（执行 1-SPP 硬件光追采样）、时空信号重建单元（执行 SVGF 时空降噪）以及光照合成单元（基于 PBR 的物理着色）。每个单元均以独立节点的身份挂载于渲染调度模块中，共同完成从几何到光影的转换。

4.2.6 交互调试模块

该模块位于架构顶端，旨在为渲染算法开发者提供一个非侵入式、高保真的辅助分析环境。其功能设计划分为两大核心维度：一是“算法实时反控”，允许用户在前端控制面板动态调整核心参数（如光追采样数、SVGF 降噪时空权重、曝光度等），并即刻反馈于最终画面，实现参数的闭环调优；二是“管线性能侦测”，通过在 RenderGraph 的关键节点埋点插入 Vulkan 时间戳查询（Timestamp Queries），实时捕捉并可视化输出 GPU 计算节点的微秒级耗时明细，为混合管线的算力瓶颈定位提供最权威的数据支撑

4.3 混合渲染管线流程设计

在明确了引擎的模块化分工后，本节将深入探讨由渲染调度模块驱动的核心业务逻辑——混合渲染管线（Hybrid Rendering Pipeline）。该管线的设计目标非常明确：打破

传统前向渲染在处理复杂光照时的效率瓶颈。我们通过构建一条“光栅化采集几何、光线追踪计算光影”的混合通道，在维持实时帧率的前提下，尽可能还原真实的物理效果。

整个混合管线在宏观架构上被解耦为四个依次递进的阶段（如图 4.4 所示）。每一个阶段的输出，都会成为后一个阶段的计算先验条件。

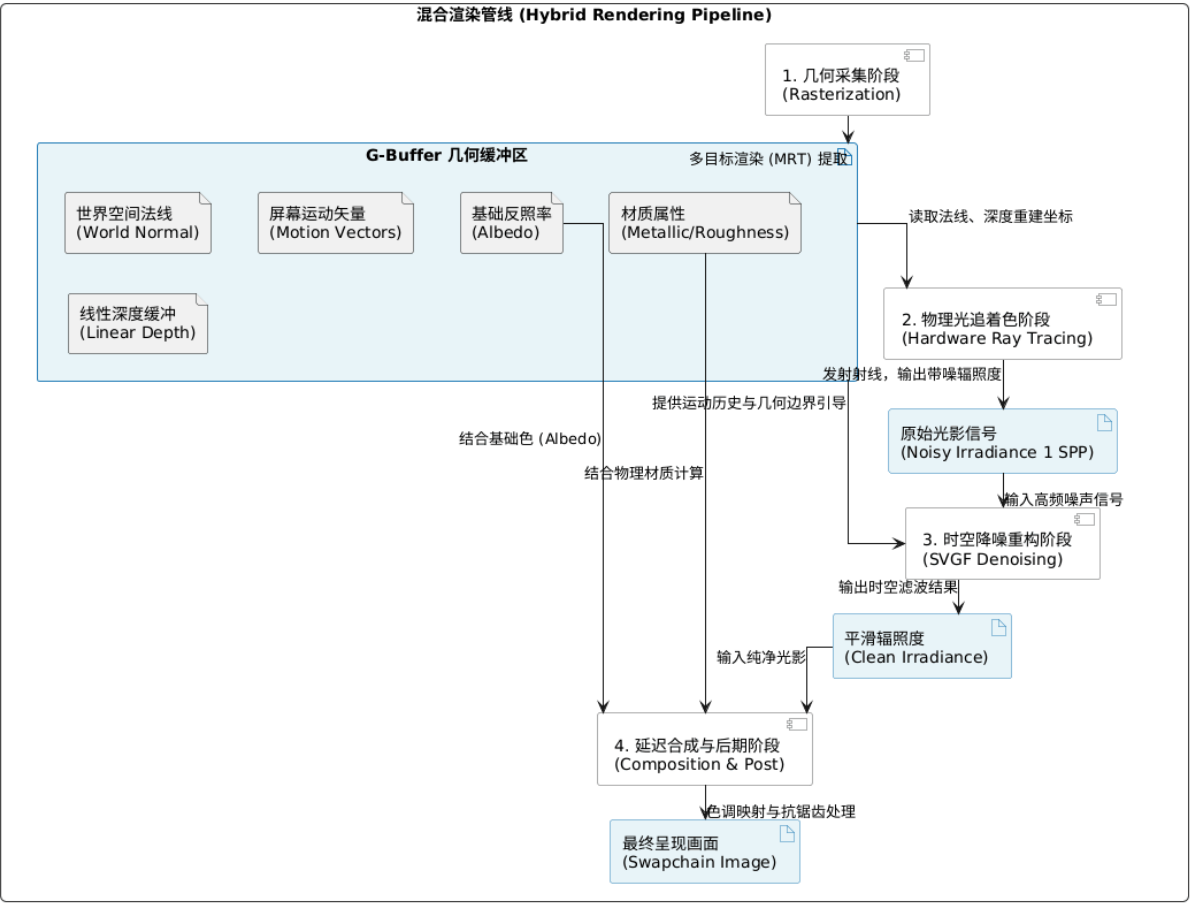


图 4.4 混合渲染管线数据流转图

4.3.1 几何特征采集

管线的起点是建立对视口的屏幕空间几何认知。在这个阶段，系统利用光栅化的高效特性，只负责剥离场景的几何体信息和材质属性，而不进行复杂的光照计算。具体来说，引擎会将深度、法线、基础反照率以及运动矢量（Motion Vectors）分别存储在不同的 G-Buffer 附件中。通过这种方式，我们成功将“找出物体在哪”和“算出物体多亮”这两件事彻底解耦。这种做法大幅降低了后续光线追踪阶段的搜索压力，因为它只需要处理屏幕可见的像素片段。

4.3.2 物理光影计算

获取屏幕空间信息后，管线正式进入光影解算的环节。系统会利用采集到的表面法线和位置数据，从当前像素点向光源或反射环境发射探测射线。考虑到实时渲染的性能预算，本项目采用了 1-SPP（单像素单样本）的低采样率策略。这种“以频率换时间”的做法虽然保证了帧率，但不可避免地会产生严重的蒙特卡洛噪声。因此，本阶段输出的原始光照信号是布满噪点的“半成品”，需要后续算法进行修补。

4.3.3 时空信号重建

为了消除低采样带来的画面瑕疵，管线引入了信号重建模块。该模块集成了 SVGF 算法。它会复用第一阶段提取的运动矢量，把当前像素与前几帧的历史数据进行对齐和累积。随后，系统会结合当前帧的几何特征（如深度突变和法线差异），执行多轮边缘保留的空间滤波。这种时空结合的策略，能在极低的开销下，将原本破碎的光影信号还原成平滑、自然的物理质感，使其在视觉上等效于高采样率的渲染结果。

4.3.4 光照合成与后期处理

在获取纯净的光影分量后，管线进入最终的合成环节。合成模块基于 PBR 物理模型，将材质的基础颜色与重构后的漫反射、镜面反射光影进行合并。合成完毕的画面最后还会经过后期处理流水线。这里不仅包含了用于消除几何边缘走样的 TAA 技术^[22]，还集成了色调映射（Tone Mapping）算法，以确保高动态范围的渲染结果能在显示器上呈现出丰富的色彩层次。

4.4 本章小结

本章完成了引擎宏观架构的整体规划。我们通过五层分层拓扑架构确立了系统的解耦原则，并定义了图形后端、资产管理、场景加速及渲染调度等核心模块的职责。此外，本章还详细梳理了混合渲染管线从几何采集到信号重建的完整数据流向。

第五章 详细设计

在第四章的概要设计中，本文已经确立了混合渲染引擎的总体架构与宏观模块划分。第五章作为本课题的核心环节，将立足于详细设计与具体工程落地，深入探讨各子系统的底层运行机制与算法实现。

5.1 图形硬件后端模块

图形硬件后端是整个混合渲染引擎的物理基座。由于现代显式图形 API 采用了极度轻量化的驱动设计哲学，将内存管理、同步原语与多线程状态机的控制权全面移交给了应用层。为了避免底层繁琐的状态机操作反向污染上层的渲染业务逻辑，本模块通过高度封装的组件化设计模式，构建了稳定、高效且线程安全的底层图形上下文。

5.1.1 基于 volk 的动态函数指针加载机制

在静态结构设计上，本模块采用了外观模式（Facade Pattern）与系统组件化思想。系统并未将所有功能塞入一个臃肿的“上帝类”，而是设计了 VulkanContext 作为全局唯一的外观枢纽，集成了 volk^[23]的动态加载核心逻辑，实现 Vulkan API 函数的动态调度与驱动适配。该枢纽将底层复杂的物理设备、显存分配器、交换链及多线程指令池封装为互相协作的子组件，对上层屏蔽了所有 Vulkan 原生句柄的获取与管理逻辑。

以下为图形硬件后端模块的静态组件拓扑图：

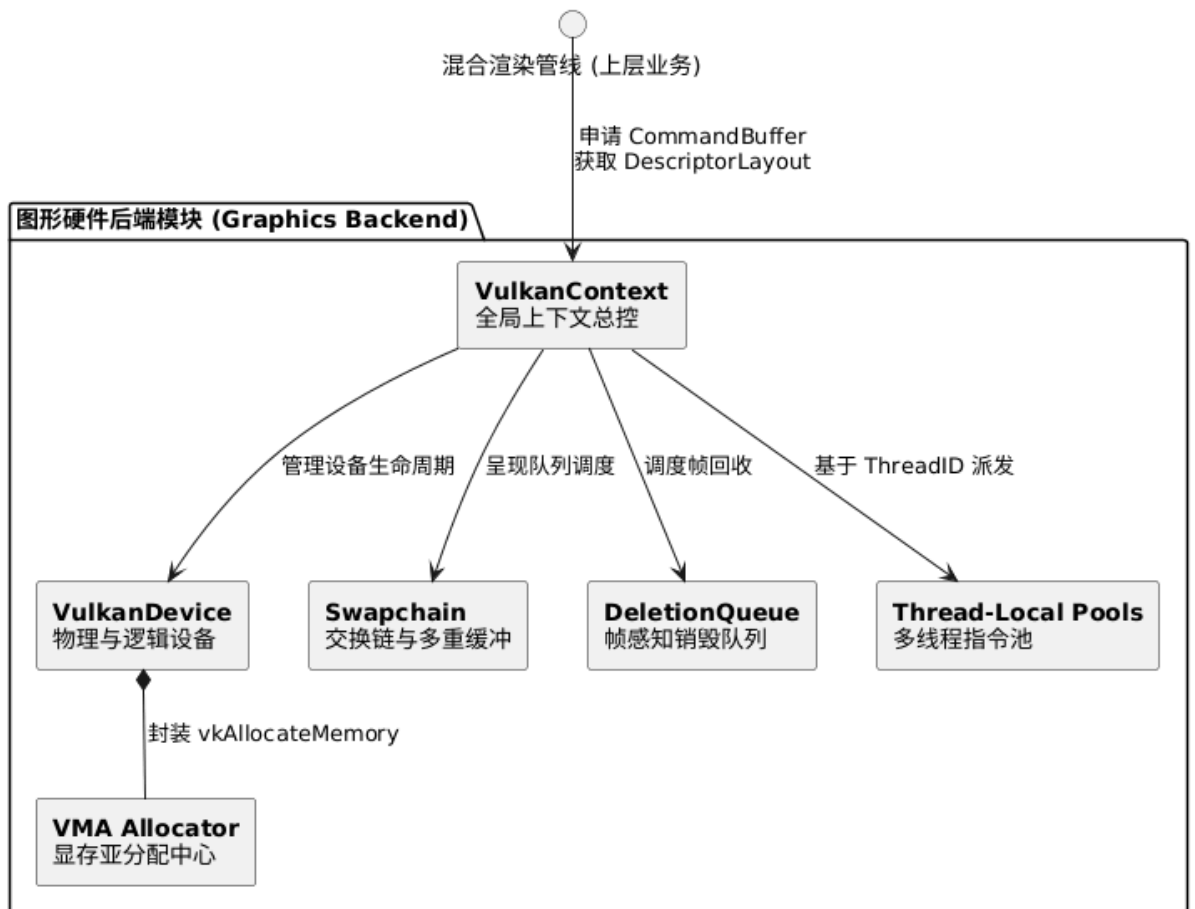


图 5.1 图形硬件后端模块静态组件拓扑图

本渲染引擎必须利用多核 CPU 的算力进行渲染指令的并行录制。然而，Vulkan 的指令池（CommandPool）在底层并不具备线程安全性。为此，系统摒弃了单指令池加锁的低效方案，在 VulkanContext 中维护了一个基于当前线程 ID（ThreadID）的动态派发组件。它为每一个异步工作线程（如主渲染线程、资源异步加载线程）动态分配独立的 CommandPool，从底层彻底消除了多线程并发录制渲染指令时的锁竞争（Lock Contention）瓶颈。

原生 Vulkan 的 vkAllocateMemory 对最大分配次数有严格的硬件上限（通常为 4096 次）。VulkanDevice 组件内部集成了 VMA。上层的任何 Buffer 或 Image 创建请求，均会被路由至该组件，由 VMA 进行大块物理显存的亚分配（Sub-allocation），有效遏制了高频资源创建带来的显存碎片化。

5.1.2 物理特性按需注入机制

混合管线对底层硬件比较挑剔。不管是 1-SPP 的硬件射线查询，还是后续要用到的

无绑定描述符，都需要特定的 GPU 扩展支持。在初始化逻辑设备时，引擎并没有把这些特性固定。我们利用 Vulkan 独特的 pNext 指针链表机制，做了一套动态的特性注入流程。

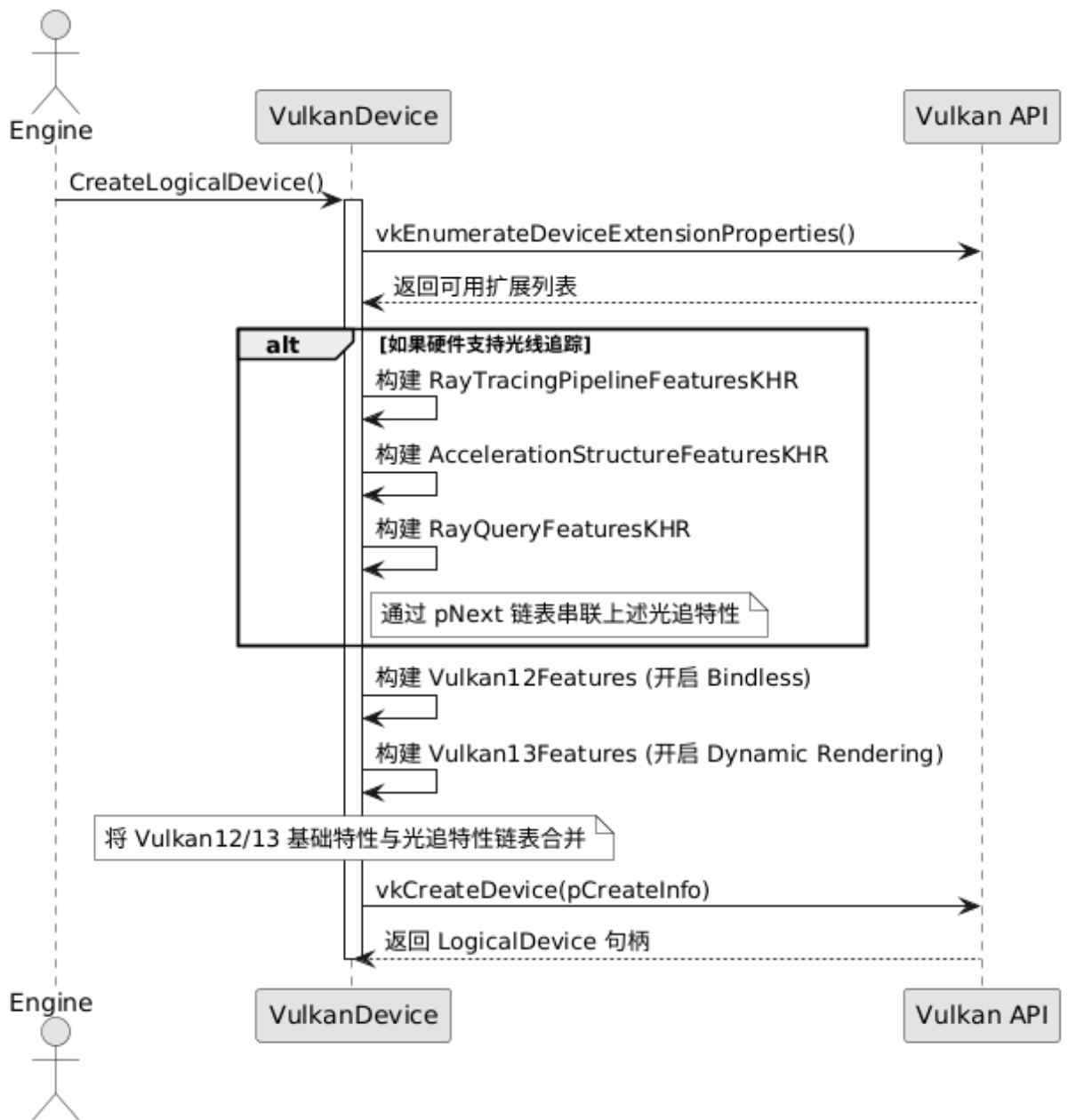


图 5.2 逻辑设备光线追踪特性动态注入时序图

启动时，系统会先探底显卡的物理算力。如果摸到了 RTX 级别的显卡，探针就会确认支持硬件光追扩展。接着，系统通过 pNext 结构体指针链，顺藤摸瓜地把光追管线（Ray Tracing Pipeline）和光线查询（Ray Query）的开启参数塞进设备上下文里。要是

碰上不支持光追的老显卡，引擎也能全身而退，安全降级到基础的光栅化管线。使用动态注入的设计，避免了因硬编码特性而导致的设备初始化崩溃，极大拉高了系统的鲁棒性。

5.1.3 交换链管理与异常恢复

本项目采用了三重缓冲架构，即 $\text{MaxFramesInFlight} = 3$ 。这就带来了一个经典的同步隐患：CPU 的指令提交进度，往往会领先 GPU 真实的渲染进度 1 到 2 帧。假设 CPU 在当前帧判定某个实体不可见，直接释放了它的显存，但此时 GPU 的命令缓冲还在上一帧辛辛苦苦地读取这个模型。这种时间轴上的错位会瞬间引发非法的内存访问，直接导致 Device Lost 级的硬件宕机。

针对此时间轴错位问题，本模块设计了帧感知延迟销毁队列（Frame-Aware Deletion Queue）机制。

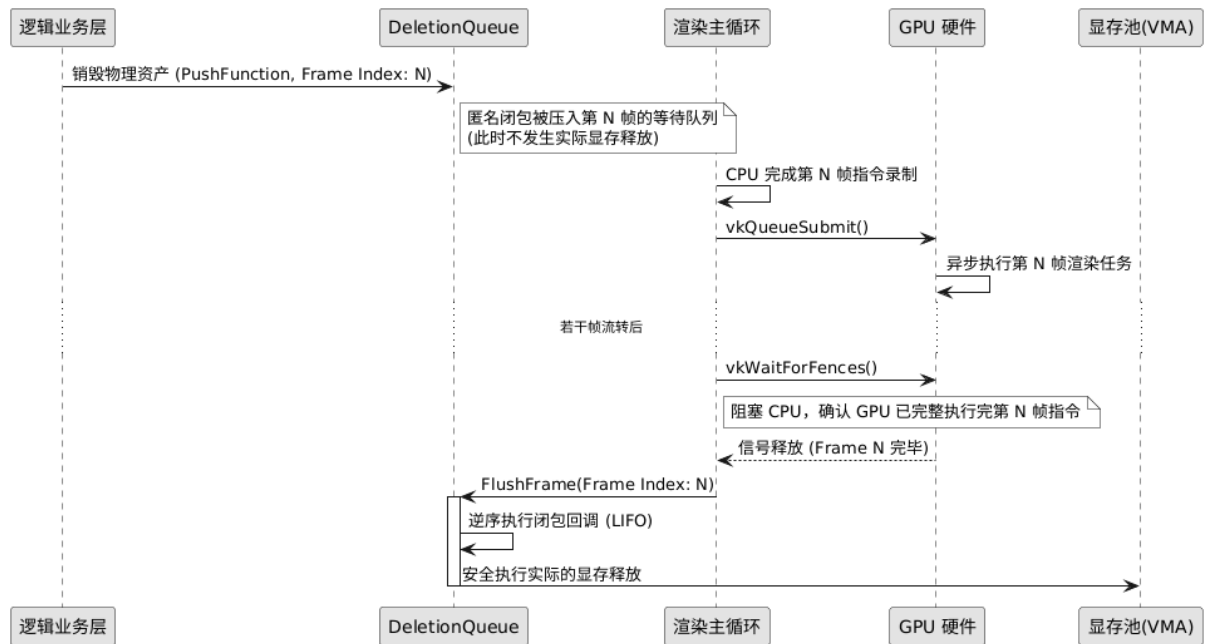


图 5.3 帧感知显存延迟回收机制时序图

5.2 资源管理模块

高精度 glTF 模型和 PBR 贴图怎么顺畅地在显存里流转是一个问题。特别是在光线追踪环节，一根射线弹射出去，不知道它最终会命中场景里的哪个物体。这就要求管线必须具备在一次 Draw Call 里随机访问全场景材质的能力。为系统引入无绑定架构^[24]，并搭配了基于 RAII 的异步引用计数机制，确保资源调度的安全性。

5.2.1 无绑定资源池

为了能让光追管线全局随机抓取贴图，**ResourceManager** 在初始化阶段，在图形后端申请超大的全局描述符数组。这里面塞满了独立的存储缓冲（SSBO）数组和采样器图像（**sampler2D**）数组。

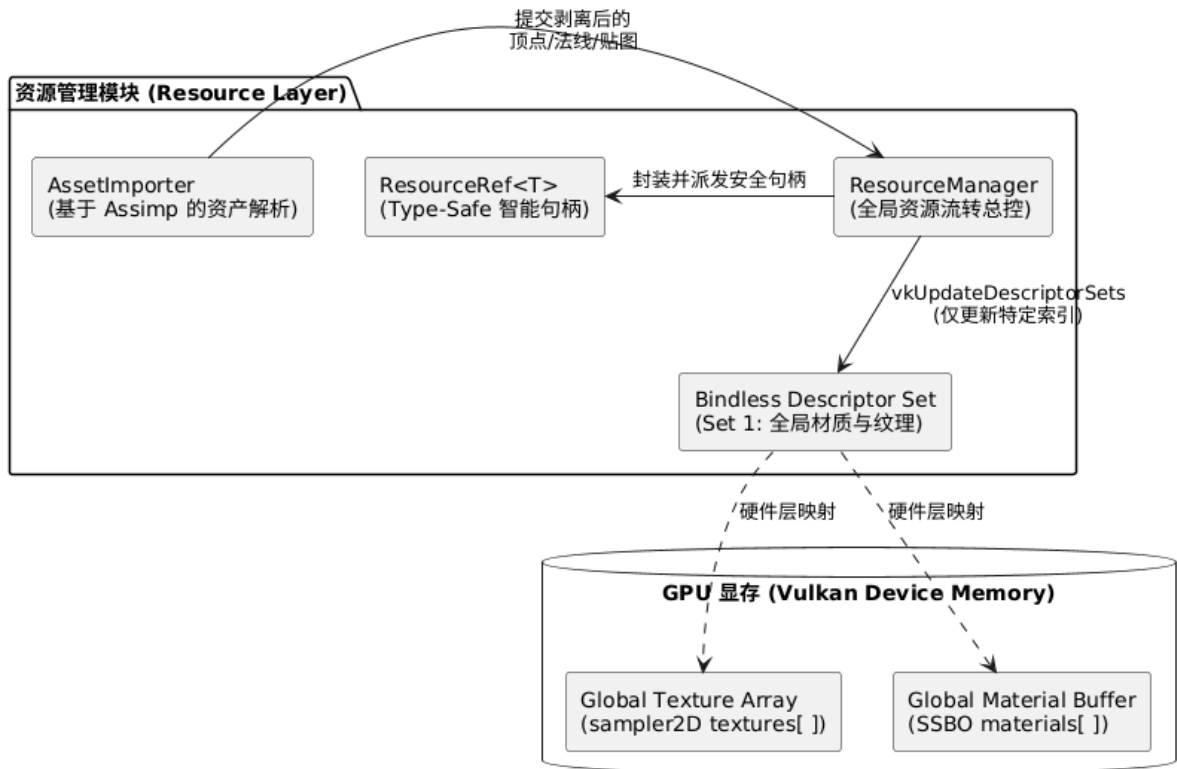


图 5.4 资源管理无绑定组件拓扑图

在这个架构下，内存里的 **Material** 对象不直接持有 **Vulkan** 的图像原生句柄。它只有一个无符号的整型索引。等到了 **Shader** 里面执行计算时，光线一旦命中某个表面，**Shader** 只要拿着这个索引去全局数组里查询，就能瞬间摸到对应的 **Albedo** 或 **Normal** 数据。

5.2.2 动态加载：基于引用计数的异步流转机制

为保证在动态加载大型外部资产时主渲染循环的流畅性，本系统在 **AssetImporter** 中实现了基于 **std::future** 的多线程异步加载机制。

异步加载的主要技术难点在于显存生命周期的并发管理。传统的显式资源释放机制在场景动态切换或实体剔除时，容易引发内存泄漏或悬空指针问题。针对此问题，本模块设计并实现了基于引用计数与延迟回收的 **ResourceRef<T>** 模板类。

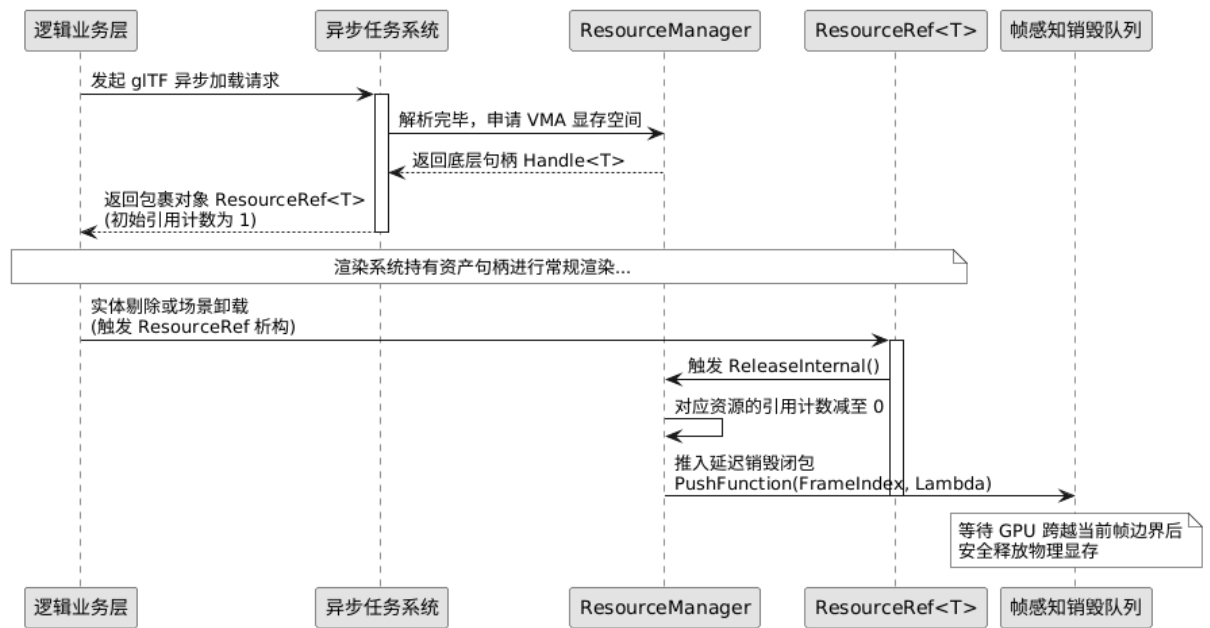


图 5.5 基于引用计数的异步资产安全生命周期时序图

借助 C++ 的 RAII 特性，渲染组件统一通过持有 ResourceRef 对象来管理资源生命周期。当某资源的最后一个引用被析构时，系统不会立刻触发硬件 API 进行显存释放，而是将释放操作推入帧感知销毁队列。该机制将实际的显存释放推移至 GPU 处理完当前帧指令之后执行，从而实现了安全的自动化显存回收管理。

5.2.3 光源预处理：蒙特卡洛积分 CDF 硬件同步

在基于物理的渲染与光线追踪管线中，计算大面积发光物体（Emissive Meshes）的软阴影通常采用下一事件估计（Next Event Estimation, NEE）重要性采样算法。由于纯随机采样会导致较高的蒙特卡洛方差，系统需要对场景中所有发光体的能量贡献进行预评估。

本模块中的 LightManager 组件承担了光源特征参数预处理与 GPU 同步的职责。

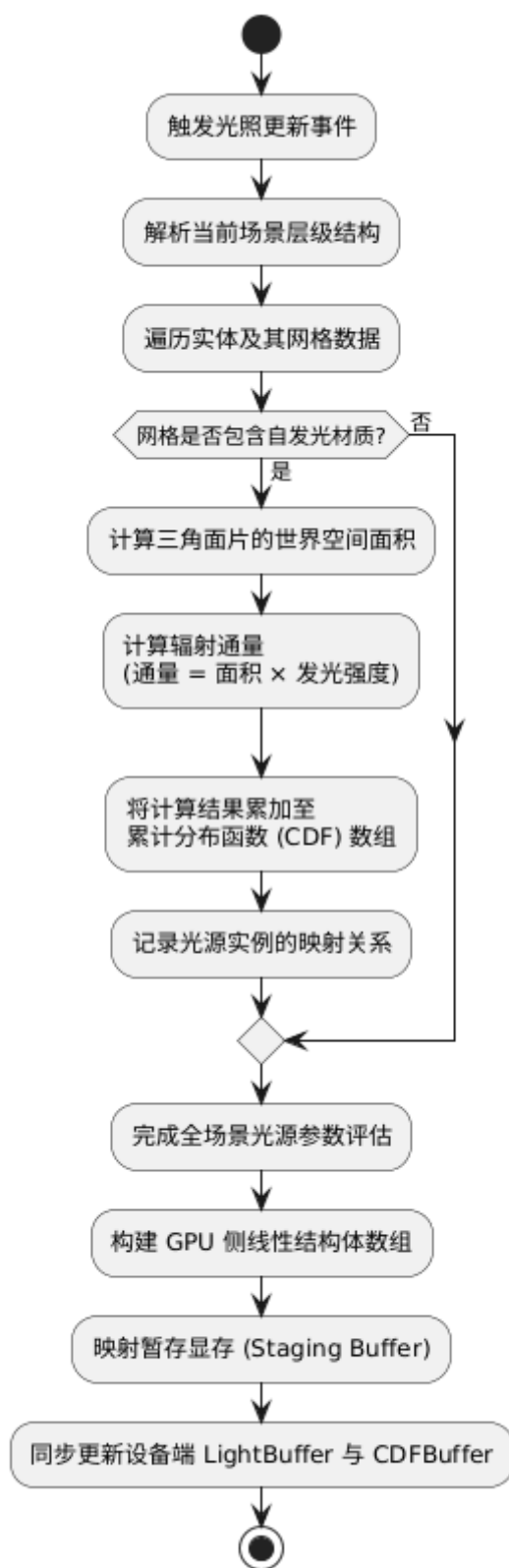


图 5.6 场景自发光体解析与 CDF 预建活动图

通过此活动流程，引擎在 CPU 端预先计算了场景中各发光三角面的概率密度分布，并构建了一维的累积分布函数（CDF）数组。将该数组同步至 GPU 后，光线追踪管线中的生成着色器（Ray Generation Shader）可通过二分查找（Binary Search）高效地根据物理能量权重选择采样光源。该前置计算步骤有效降低了光线追踪采样的初始方差，从而减轻了后续 SVGF 降噪阶段的滤波压力。

5.3 场景管理模块

场景管理模块是连接静态底层资产与动态渲染管线的核心枢纽。在混合渲染架构中，场景不仅需要处理传统的光栅化视锥体剔除（Frustum Culling），更需要实时维护光线追踪管线所依赖的三维空间拓扑（TLAS/BLAS）。同时，摄像机系统作为场景的观察者，必须为时域抗锯齿（TAA）和运动矢量的计算提供高精度的矩阵投影与抖动状态。

5.3.1 光追加速结构动态构建机制

硬件光线追踪无法直接遍历传统的顶点与索引缓冲。为了在极短的算力预算内完成光线与海量三角面片的求交测试，必须依托于 GPU 专用的双层边界体积层次结构，即底层加速结构（BLAS）与顶层加速结构（TLAS）。

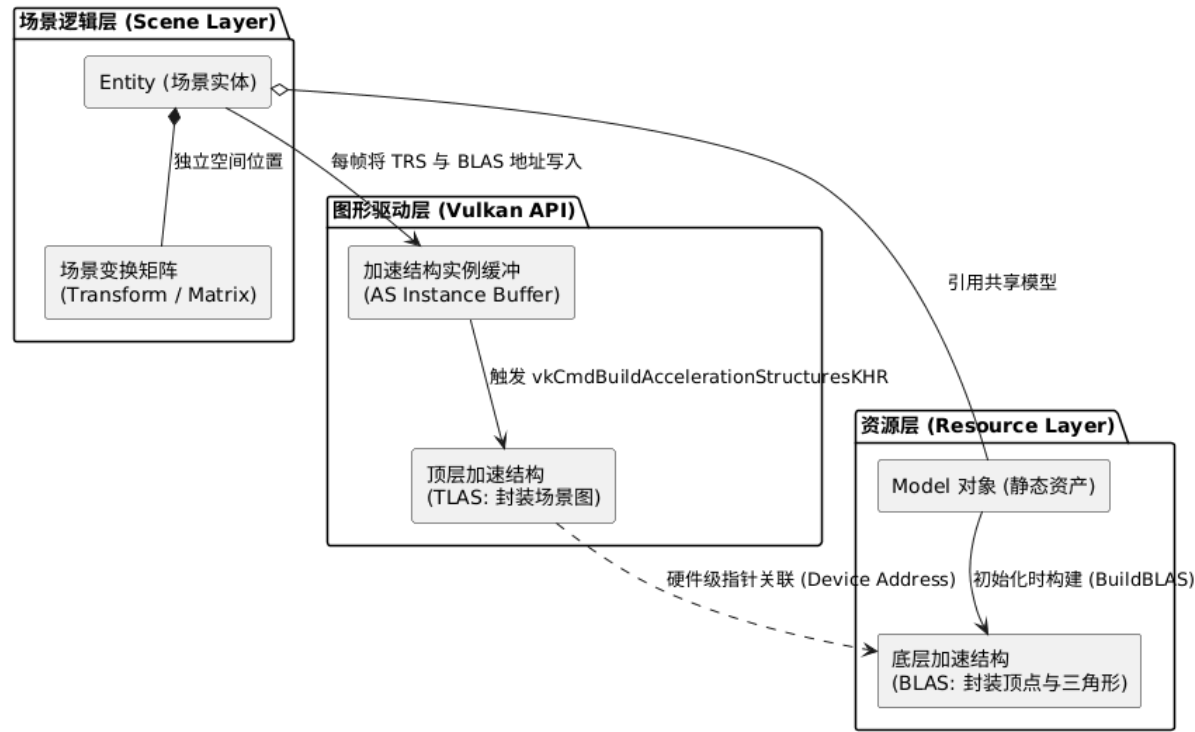


图 5.7 光线追踪双层加速结构分级构建架构图

在该机制中，Model 模块负责预编译 BLAS。由于模型拓扑结构通常是静态的，

BLAS 仅在资产加载阶段利用暂存缓冲（Scratch Buffer）在 GPU 端构建一次。而在主渲染循环中，Scene 模块负责监控所有 Entity 的空间变换矩阵。一旦实体发生位移、旋转或缩放，场景将收集其关联的 BLAS 硬件地址与新的变换矩阵，重构实例缓冲（Instance Buffer），并在每帧提交前动态更新 TLAS。这一机制确保了光线探测单元（RT Core）始终拥有绝对一致的物理世界视图，且避免了冗余的底层几何体重建开销。

5.3.2 基于八叉树的视锥体剔除策略

为了极大降低第一阶段 G-Buffer 提取时的光栅化压力，系统在 CPU 端实现了一套基于八叉树（Octree）的空间剔除方案。

针对场景中存在的大量静态与动态实体，Scene 对象会在空间初始化时构建八叉树节点网络。每个节点维护其覆盖的三维轴向包围盒（AABB）。在渲染流水线发起 GetVisibleEntities 查询时，系统采用深度优先遍历策略：将摄像机的视锥体（Frustum）与八叉树节点的 AABB 进行相交测试。若视锥体完全处于某节点外部，则该节点及其所有子节点下的 MeshComponent 将被直接抛弃，不再进入后续的 Vulkan 渲染指令录制阶段。此举在 CPU 端过滤了海量不可见几何体，有效防止了 GPU 前端顶点处理单元的拥堵。

5.3.3 时域抖动注入与历史矩阵追踪系统

现代混合渲染引擎高度依赖时间维度的数据复用。无论是 SVGF 还是 TAA，其核心基础均是精确的运动矢量。本系统对 EditorCamera 进行了深度定制，引入了反向 Z（Reversed-Z）投影与亚像素抖动（Jitter）注入机制。

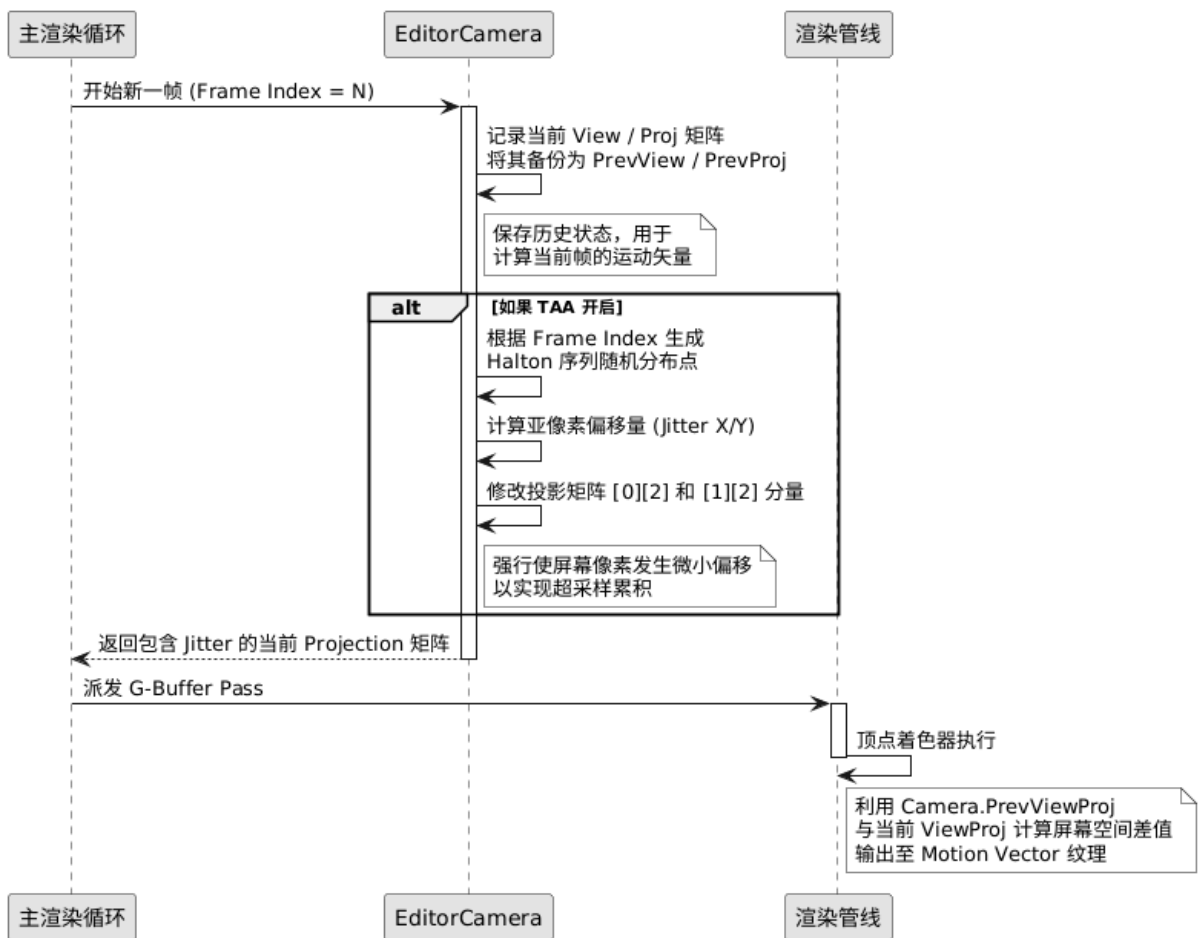


图 5.8 摄像机时域抖动注入与历史矩阵追踪时序图

1. 反向 Z 投影：由于浮点数在接近 0 时具有更高的精度，相机类在构建透视投影矩阵时，将远裁剪面（Far Clip）映射至0.0，近裁剪面（Near Clip）映射至 1.0。这一设计彻底消除了大尺度开放场景下的深度冲突（Z-Fighting）现象。
2. 亚像素抖动与速度缓冲：当 TAA 激活时，相机会利用 Halton 序列生成分布均匀的二维偏移量，并动态注入到透视投影矩阵的第三列分量中^[25]。同时，相机必须严格记录上一帧的视图投影矩阵（PrevView 与 PrevProj）。在 G-Buffer 的顶点着色器阶段，顶点会同时通过当前帧矩阵与历史帧矩阵进行变换，两者的屏幕空间坐标差值即为真实的运动矢量。这是指引 SVGF 降噪算法正确“回溯历史”的唯一数学依据。

5.4 渲染调度模块

渲染调度模块是引擎驱动复杂混合管线的核心中枢。由于本项目涉及光栅化、硬件光线追踪及多级计算着色器（Compute Shader）的交替执行，各阶段间的资源读写依赖极其复杂。若采用传统的硬编码同步方案，不仅开发效率低下，且极易引发竞态条件

（Race Condition）。

针对此问题，本模块实现了基于 DAG 的渲染图系统。该系统通过声明式的资源依赖描述，实现了同步屏障的自动推导、资源生命周期的智能管理以及异构计算任务的统一调度。

5.4.1 任务编排与资源依赖的声明式建模

RenderGraph 的核心思想是将渲染过程抽象为一系列 Render Pass 与资源的组合。本系统通过 PassBuilder 接口，强制要求每个 Render Pass 在执行前声明其对资源的读写意图。

以下为渲染调度模块的组件交互与依赖建模设计：

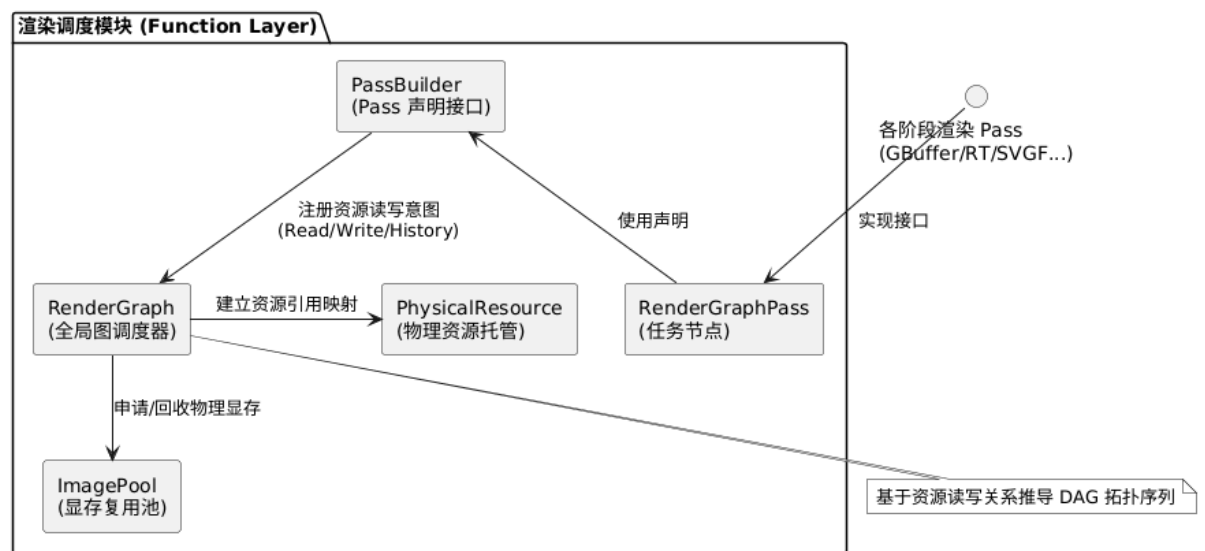


图 5.9 RenderGraph 资源依赖与任务编排组件设计图

5.4.2 自动同步屏障推导与状态转换机制

Vulkan 1.3 引入了 Synchronization2 扩展，极大简化了同步原语。本模块充分利用这一特性，在 RenderGraph 内部实现了一套自动化的资源状态自动机。当一个 Pass 声明对某资源进行“写入（Write）”而后续 Pass 声明“读取（Read）”时，系统会自动在指令流中插入精确的内存屏障。

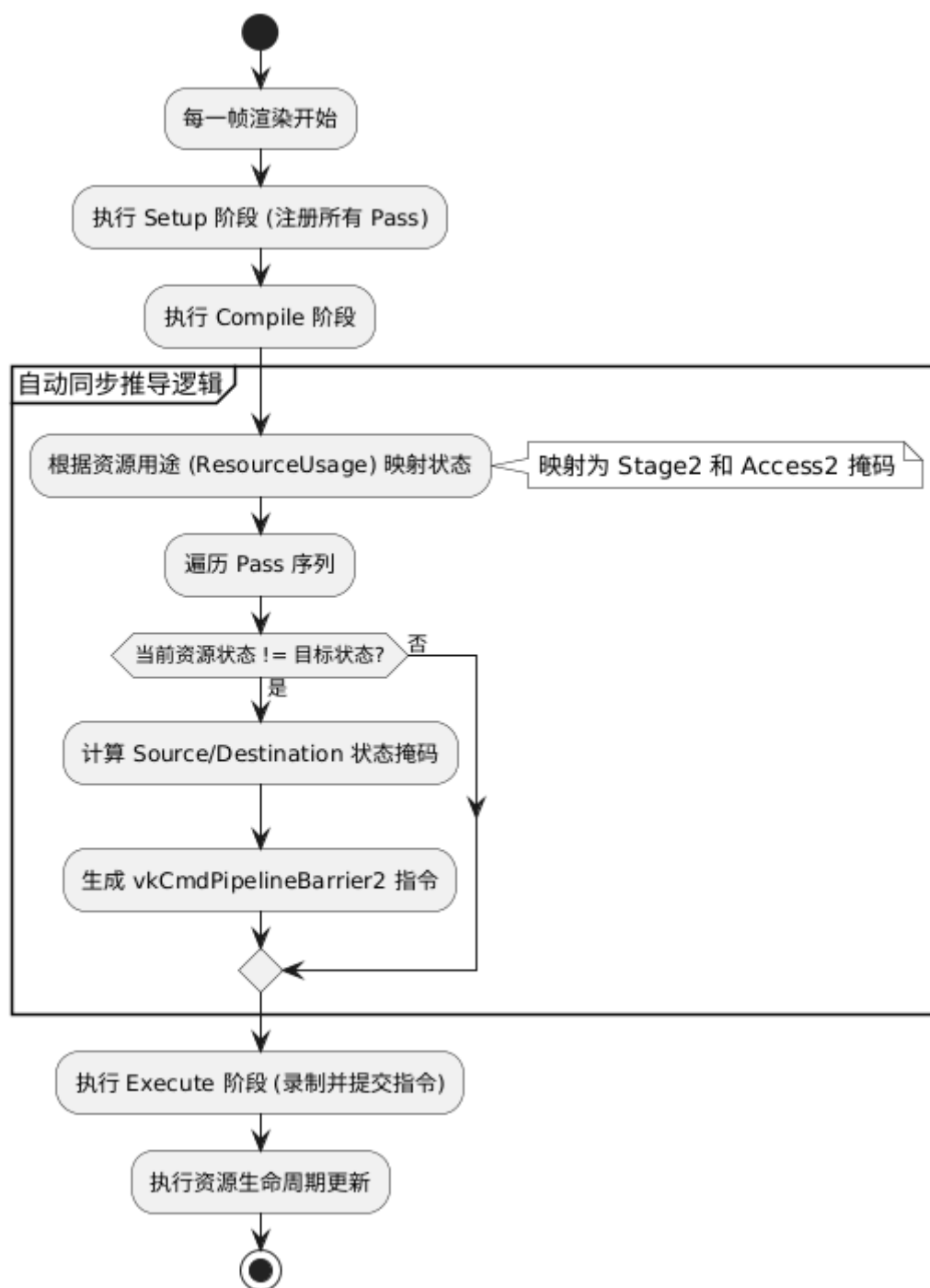


图 5.10 资源状态自动机与同步屏障自动生成活动图

该机制将复杂的 `VkPipelineStageFlags2` 和 `VkAccessFlags2` 掩码计算逻辑封装在调度层内部。例如，当光线追踪阶段（Ray Tracing Stage）输出的阴影图需要进入 SVGF 降噪阶段（Compute Stage）作为采样输入时，系统会根据预设的 `GetStateFromUsage` 逻辑，自动推导出从 `SHADER_STORAGE_WRITE` 到 `SHADER_READ_ONLY` 的布局转

换及对应的缓存刷新操作，确保了 GPU 端数据的可见性。

5.4.3 异构计算任务的执行上下文封装

为了提升渲染逻辑的可读性并降低 Vulkan 描述符集（Descriptor Sets）管理的复杂度，系统针对不同类型的硬件计算任务设计了统一的执行上下文（Execution Context）体系。该体系通过对底层指令缓冲（CommandBuffer）的二次封装，为光栅化、通用计算及硬件光线追踪提供了领域专用接口。

在具体设计上，系统实现了接口的专业化与抽象化。针对不同的管线需求，引擎提供了定制化的上下文实现：例如，GraphicsExecutionContext 封装了视口设置、顶点缓冲绑定及光栅化流水线绑定等图形接口；ComputeExecutionContext 专注于计算核函数的派发（Dispatch）；而 RaytracingExecutionContext 则负责空间加速结构的绑定与光线追踪指令（TraceRays）的下发。这种层次划分使得具体的算法实现能够与底层的 Vulkan 原生 API 调用相解耦，从而提升了业务代码的可维护性。

同时，为简化描述符管理流程，该体系集成了着色器反射（SPIR-V Reflect）数据，实现了资源绑定的自动化。在 Render Pass 执行期间，上下文会动态解析当前着色器的资源布局信息，完成描述符集的更新与绑定工作。开发者只需通过抽象的资源句柄指定管线的输入输出，上下文模块便会在后台自动处理绑定偏置（Binding Offsets）与描述符池的分配逻辑。此外，为实现异构任务的统一调度，所有类型的执行上下文均继承自同一基类，共享一致的推入常量（Push Constants）下发机制与底层指令缓冲管理逻辑。这种统一的底层抽象确保了光栅化与光线追踪任务在 RenderGraph 调度器中具备标准化的接口规范，从而支持了跨阶段渲染管线的协同执行与计算资源的集中调度。

5.5 混合渲染管线模块

在完成了底层的资源调度与空间结构管理后，本节将重点论述混合渲染管线的核心算法实现。混合管线旨在结合光栅化的高效几何处理能力与硬件光线追踪的物理精确性。系统将整体算法拆解为几何特征提取、低采样率光影探测以及时空降噪滤波三个依次递进的 Render Pass。

5.5.1 基于光栅化的几何特征提取

混合管线的首个阶段是剥离场景的几何与材质信息，避免在后续昂贵的光线追踪阶段发生无意义的过度绘制（Overdraw）。本系统利用光栅化管线提取屏幕空间的可见表

面数据，并将其压缩存储至多张渲染目标(Render Targets)中，构成几何缓冲(G-Buffer)。

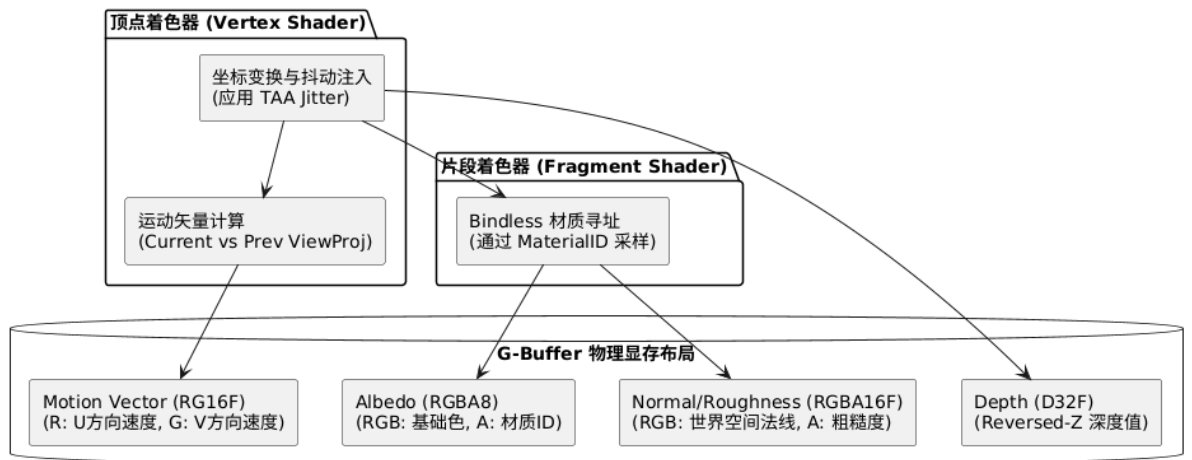


图 5.11 几何特征提取阶段与 G-Buffer 显存布局图

在此阶段，系统为支持 TAA 与时空降噪 SVGF，在顶点着色器中引入了历史矩阵追踪机制。顶点不仅通过当前帧的视图投影矩阵计算屏幕坐标，同时利用上一帧的历史矩阵计算前置屏幕坐标。两者的屏幕空间坐标差值被量化并输出至运动矢量纹理（Motion Vector Texture）中。此外，法线数据被映射至高精度的 RGBA16F 格式以防止量化误差导致的光线反射偏移，而深度缓冲则采用反向 Z 映射以保障远距离视角的深度测试精度。

5.5.2 单样本光线追踪探测

在获取了屏幕空间的几何属性后，管线进入光线追踪探测阶段。受限于实时渲染的毫秒级算力预算，系统无法采用离线渲染中高频率的蒙特卡洛积分（Monte Carlo Integration）。本模块实现了基于每像素单次采样（1-SPP）的硬件光追策略，主要用于计算精确的软阴影与环境光遮蔽（AO）。

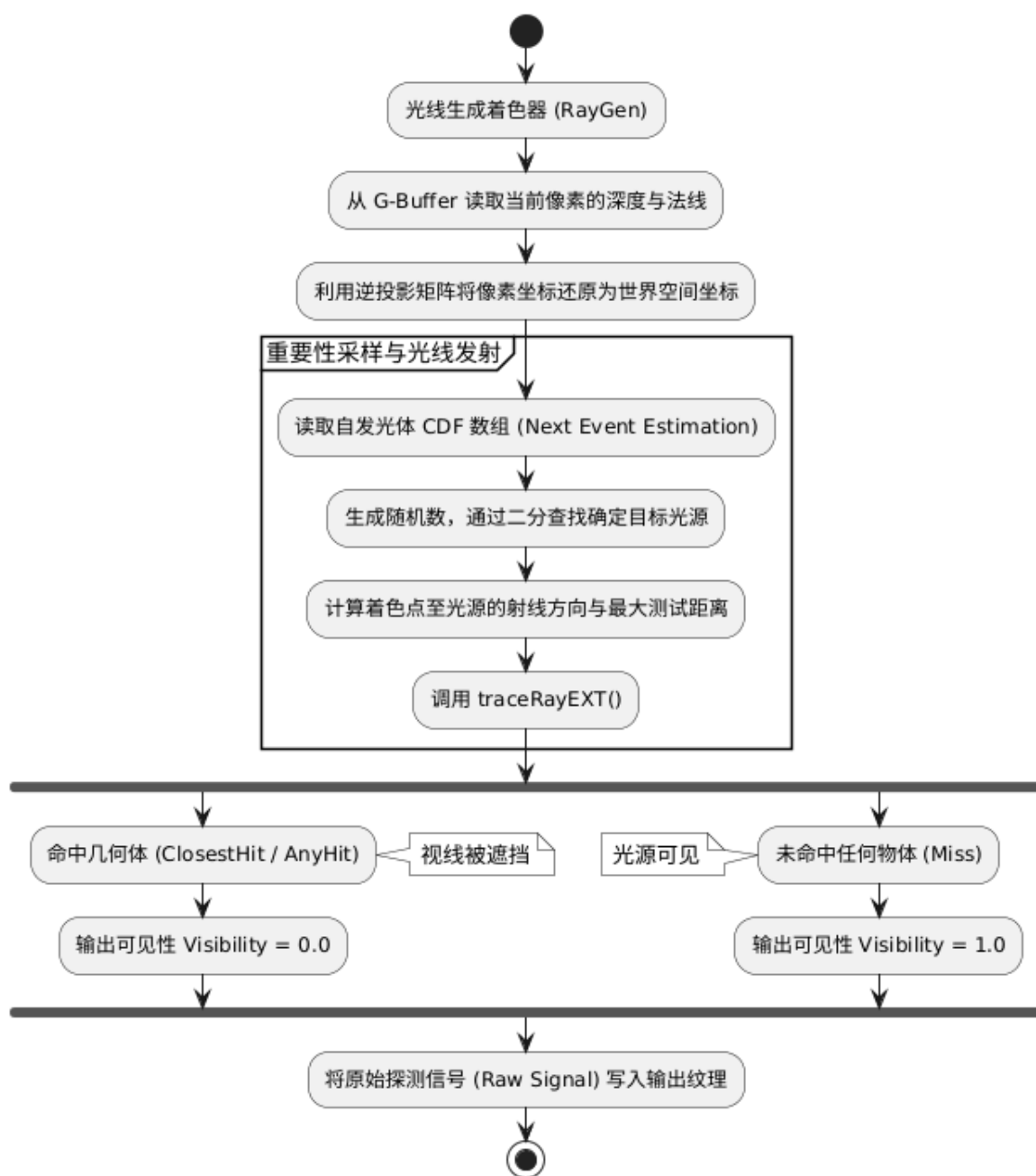


图 5.12 1-SPP 硬件光线追踪软阴影探测活动图

在光线生成（RayGen）着色器中，算法首先根据 G-Buffer 还原像素的三维世界空间坐标。为了降低 1-SPP 带来的严重方差（噪点），系统利用了 5.2 节构建的光源累积分布函数（CDF）进行下一事件估计（NEE）。算法倾向于向能量贡献度更高的自发光体发射测试光线，从而在极低的采样率下获取更加可靠的初始光影信号。探测结果（被遮挡或可见）以原始噪声纹理的形式输出，作为降噪器的输入先验。

5.5.3 SVGF 降噪实现

由于 1-SPP 探测输出的信号包含剧烈的高频噪声，无法直接用于最终画面合成。本模块引入了时空方差引导滤波（Spatiotemporal Variance-Guided Filtering, SVGF）算法。该算法通过时间维度的历史累积与空间维度的自适应模糊，重构出纯净的物理光影。

算法的执行流程被拆分为三个顺序执行的计算着色器（Compute Shader）通道：

1. 时域重投影与累积 (Temporal Reprojection): 算法首先利用几何阶段输出的运动矢量，在历史帧缓冲中查找当前像素的前置状态。系统通过比较当前帧与历史帧的深度差与法线夹角来判定历史数据的有效性。若判定有效，则根据指数滑动平均（EMA）模型混合当前帧信号与历史帧信号；若判定为几何遮挡或场景切换导致的历史失效，则丢弃历史数据，将累积历史帧数重置为 0。
2. 空间方差估计 (Variance Estimation): 时间累积能够有效降低静态区域的噪声，但在运动物体边缘或新出现的区域，因历史数据重置，局部方差依然极高。此通道通过计算像素周围 7×7 邻域内的光照一阶矩与二阶矩，估算出当前像素的统计学方差。该方差将作为后续空间滤波器的引导权重。
3. 自适应 A-Trous 空间滤波 (A-Trous Spatial Filtering): 针对高方差区域，系统采用多级 A-Trous 小波变换进行空间模糊。为了在抹平噪点的同时保留尖锐的几何边缘与阴影边界，滤波器引入了基于 G-Buffer 属性的联合双边过滤权重（Edge-stopping Function）。其权重函数 w 综合了深度、法线与光照亮度（Luminance）的差异：

$$w = \exp \left(-\frac{|\Delta z|}{\sigma_z} - \frac{\arccos(N_1 \cdot N_2)}{\sigma_n} - \frac{|\Delta L|}{\sigma_L \sqrt{V}} \right) \quad (5-1)$$

其中 Δz 与 N 分别代表深度差与法线， ΔL 代表亮度差， V 为上一步估算的方差。方差 V 越大，亮度权重项的惩罚越小，模糊程度越深。通过多次迭代不同步长的 A-Trous 滤波，算法最终以较低的性能开销实现了大范围的噪声平滑。

在经过上述几何采集、物理探测与时空降噪后，纯净的光影信号将与基础反照率进行 PBR 物理合成，最终进入后期处理管线进行色调映射与输出。

5.5.4 光照合成与后期处理管线

在 SVGF 阶段完成后，管线获取了无噪点的纯净物理光影信号。然而，此时的数据仍处于高动态范围（HDR）的线性色彩空间中，且缺乏材质的基础表面属性。为了输出最终的设备可视图象，系统设计了光照合成与后期处理管线，该管线由三个依次递进的

着色器阶段构成。

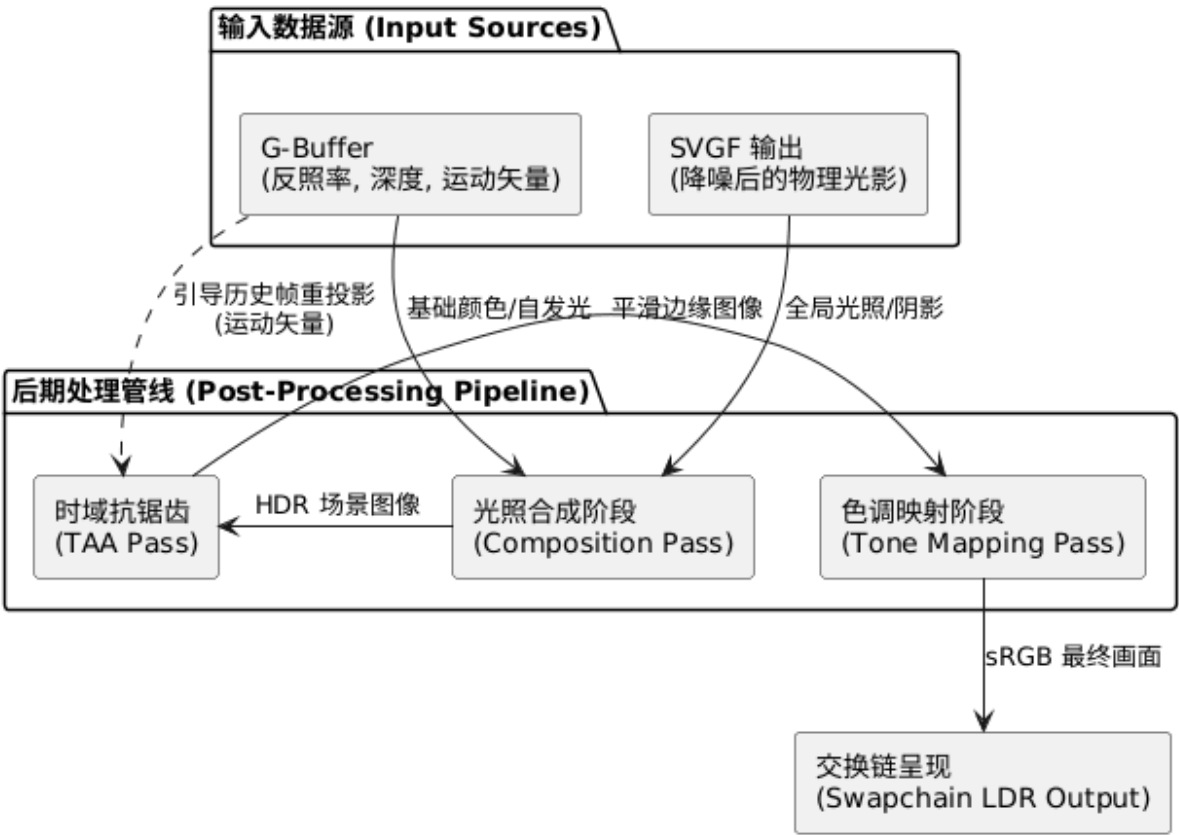


图 5.13 光照合成与后期处理数据流转图

1. 物理光照合成 (Composition): 合成着色器首先从 G-Buffer 中采样反照率与自发光 (Emission) 参数，并将其与 SVGF 输出的漫反射 (Diffuse) 及镜面反射 (Specular) 光影进行数学乘加运算。在此阶段，系统还会根据环境映射参数融合天空盒 (Skybox) 的背景辐射度，从而还原出真实的物理材质质感。
2. 时域抗锯齿: 虽然 SVGF 解决了蒙特卡洛积分的方差噪点，但光栅化阶段产生的几何边缘阶梯状锯齿依然存在。TAA 通道利用几何提取阶段注入的亚像素抖动，结合运动矢量对历史帧进行重投影累积。算法通过对当前帧邻域像素的色彩空间 (如 YCoCg 空间) 进行方差限制 (Variance Clipping)，有效抑制了历史数据重投影时常见的“鬼影 (Ghosting)”伪像，实现了极低开销的高质量边缘平滑。
3. 色调映射与伽马校正 (Tone Mapping & Gamma): 由于 HDR 图像的亮度分布远超常规显示设备的色域上限，最终的后期处理通道引入了色调映射算法 (如 ACES 拟合曲线)。该算法对极高亮度的像素进行平滑的高光压制 (Highlight Compression)，防止画面过曝丢失细节；随后执行标准的伽马校正 (Gamma

Correction），将线性空间的物理光照转换为 sRGB 色彩空间，最终交由图形界面的交换链进行屏幕呈现。

此阶段的执行标志着单帧渲染管线数据流转的彻底结束。

5.6 交互调试模块

在实时渲染引擎的开发与优化过程中，硬编码（Hard-coding）参数往往会导致极低的迭代效率。为了向图形算法开发者提供非侵入式的分析环境，本系统在渲染管线之上构建了交互调试与工具层模块。该模块基于层叠式架构（Layer Stack）与即时模式图形界面（Dear ImGui），实现了应用生命周期的统一托管、渲染参数的闭环调优以及底层 GPU 性能的可视化侦测。

5.6.1 应用生命周期与事件驱动机制

为了提升开发效率，系统实现了一套基于事件驱动的参数调节机制，在该数据反馈闭环中，引擎的应用层以 Application 类为核心枢纽，负责接管操作系统的窗口事件、统筹各个功能层（Layer）的更新，并驱动渲染主循环。为实现业务逻辑与底层图形后端的解耦，系统采用了基于层级栈的事件路由机制。

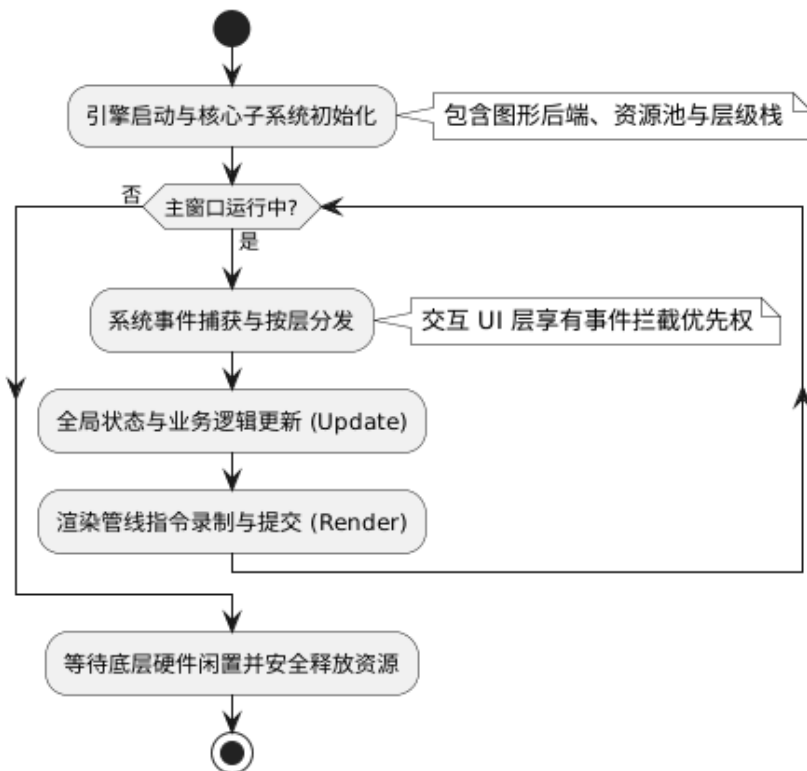


图 5.14 渲染参数动态调节与 UBO 更新顺序图

在该机制下，用户的键鼠输入首先由操作系统捕获并转化为标准事件，随后按层级栈从顶至底传递。当用户在交互界面（如控制面板）上操作时，`ImGuiLayer` 会拦截该事件，防止其向下穿透触发场景相机的误操作。这种设计保证了调试界面与场景漫游逻辑的互不干扰。

5.6.2 非侵入式参数反控与闭环调优

混合渲染管线中的诸多参数（如曝光度、降噪器的时空权重、采样率等）直接影响最终的画面质量与执行效率。为实现非侵入式的参数调优，系统建立了从前端 UI 到 GPU 显存的数据同步通路。

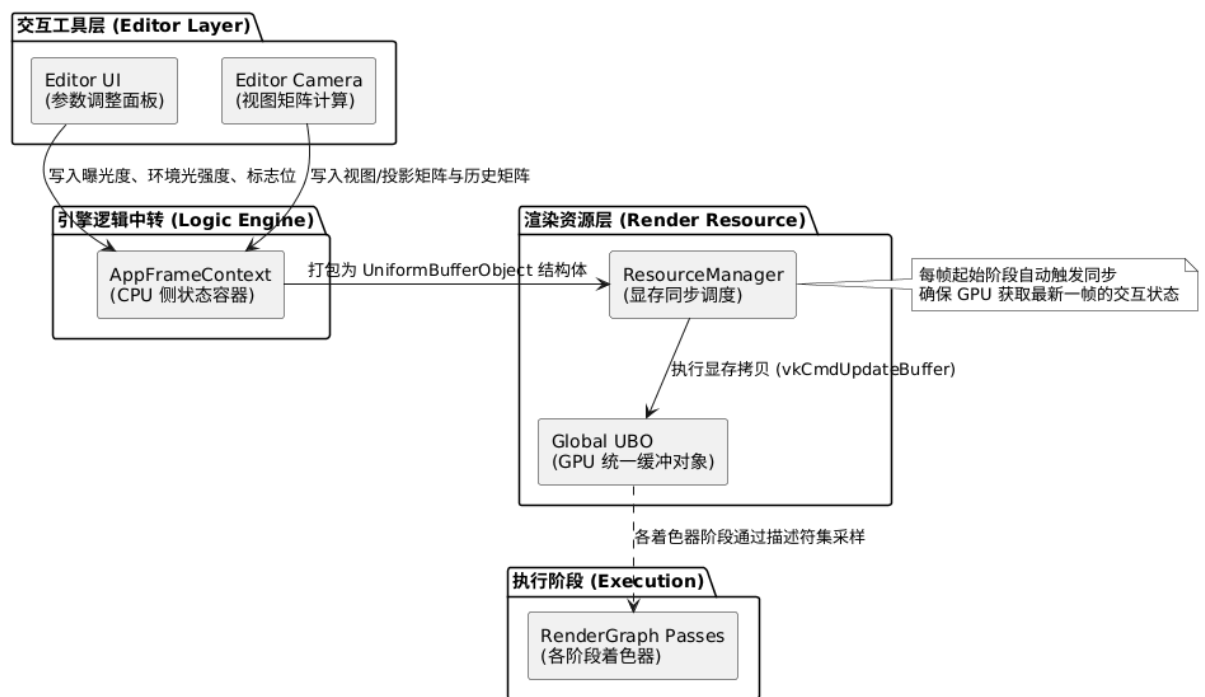


图 5.15 渲染参数实时控制与显存同步数据流向图

在该机制下，交互层产生的参数变更首先被记录在 CPU 端的 `AppFrameContext` 状态容器中。在每帧的更新阶段，系统通过 `ResourceManager` 将这些散列的状态量打包成符合显存对齐规范的 `UniformBufferObject` 结构体，并利用 Vulkan 的显存拷贝指令同步至全局统一缓冲（UBO）。这种设计实现了交互数据与渲染管线的解耦，支持在不重新编译着色器的情况下实现渲染效果的即时反馈。

5.6.3 图形界面后端与性能可视化

为了支持对复杂渲染管线的深度审查，系统实现了针对 Vulkan 后端的图形界面适

配，并集成了性能侦测模块。

在资源展示层面，交互层通过独立分配的描述符池，实现了渲染目标的动态挂载。开发者可以将 `RenderGraph` 在各中间阶段生成的图像（如 `G-Buffer` 的法线图、光线追踪产生的原始阴影图等）直接映射至 UI 窗口中进行可视化审查。这种能力对于定位混合管线中的数据冒险或算法逻辑错误具有重要作用。

在性能监控层面，系统利用硬件提供的时间戳查询（`Timestamp Queries`）机制，在渲染任务图的每个通道起点与终点插入埋点。系统采集这些微秒级的耗时数据，并在交互面板中以图表形式实时展示各渲染阶段的 GPU 算力分布。这为衡量降噪算法的开销、平衡光线追踪采样率与实时性之间的关系提供了客观的数据依据，支持了渲染管线的针对性优化。

5.7 本章小结

本章完成了系统六大核心模块的底层逻辑设计。通过对调度算法、资源模型及信号重建链路（`SVGF/TAA`）的深度拆解，形成了严密的详细设计方案，为下一章的系统实现奠定了坚实的工程基础。

第六章 系统实现与测试

经过前五章的架构设计与底层编码，本章是整个混合渲染引擎的跑通与验收阶段。本章记录了引擎在真实物理硬件上的运行表现。将结合实际抓取的运行截图，依次展示前端控制台、中间生成的 G-Buffer，以及 SVGF 降噪前后肉眼可见的画质差异。随后，利用设定好的测试用例，检验底层显存分配与 RenderGraph 调度的抗崩溃能力。

6.1 系统运行环境

操作系统：Windows 11；

开发软件：Visual Studio Code；

软件环境：C++ 20 Vulkan SDK (兼容 1.3/1.4 规范)；

图形处理器 (GPU)：NVIDIA GeForce RTX 5070 Ti Laptop GPU (12GB VRAM)

6.2 系统功能实现

本节将严格按照渲染管线的数据流向，直接贴出各个核心执行阶段的物理输出画面。借此直观地论证前文推导的光追与降噪算法是否真正在工程中生效。

6.2.1 交互式调试界面展示

为了避免在底层代码里反复写死参数进行调试，系统接入了轻量级的 ImGui 面板。从图 6.1 可以看出，控制台左侧显示当前场景物体加载模型。右侧是一系列核心参数的控制接口。开发者可以直接用鼠标实时切换渲染路径或调整光照参数，同时画面更新。

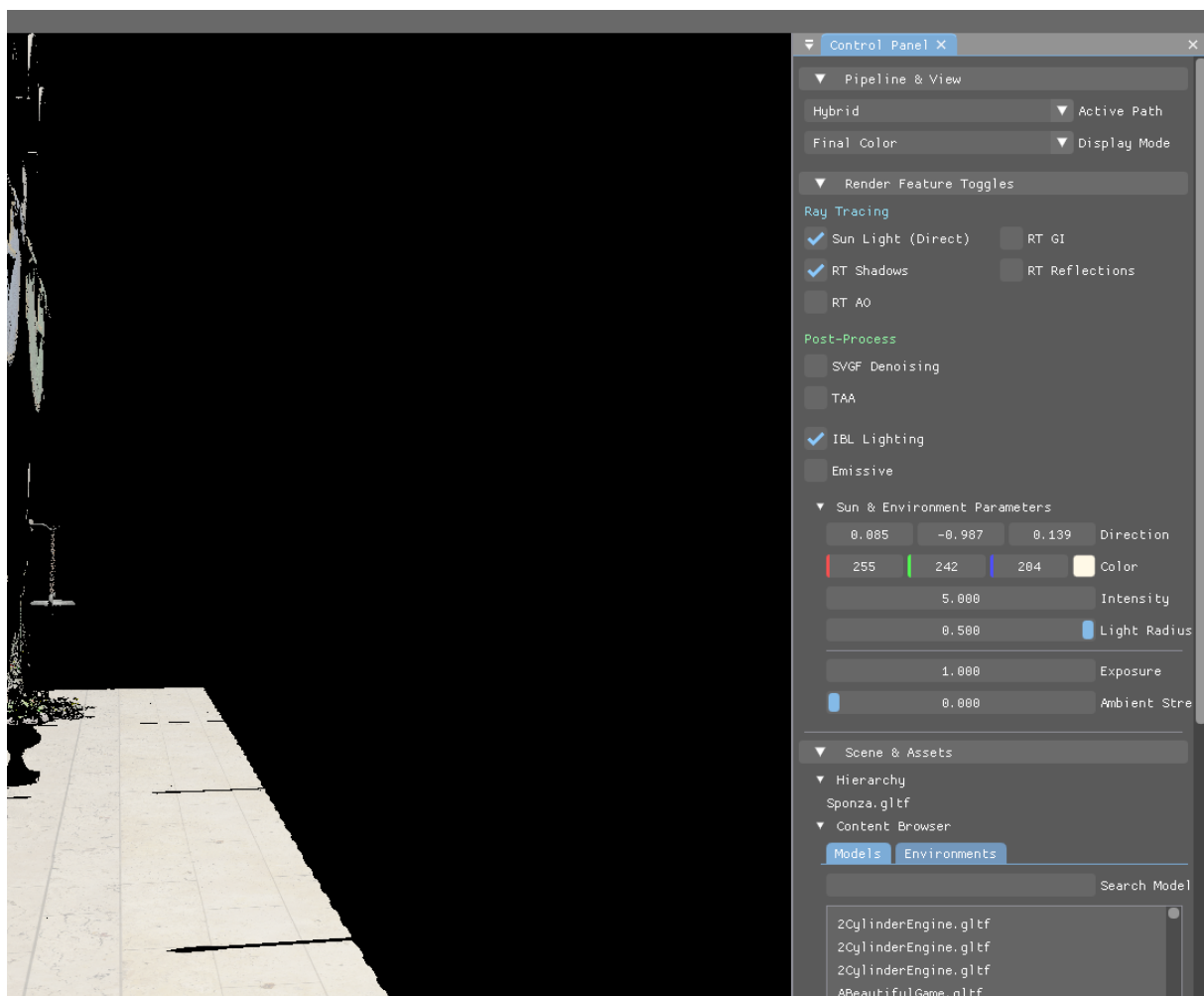


图 6.1 交互面板图

6.2.2 G-Buffer 几何特征可视化

几何数据的精准提取，决定了后续的光线追踪会不会发生错位。利用工具层的 Render Pass 可视化功能，我们可以直接把屏幕切换到特定的调试视图。图 6.2 清晰地展示了光栅化阶段输出的各项物理属性，包括屏幕空间深度（Depth）、世界空间法线（Normal）以及基础反照率（Albedo）。

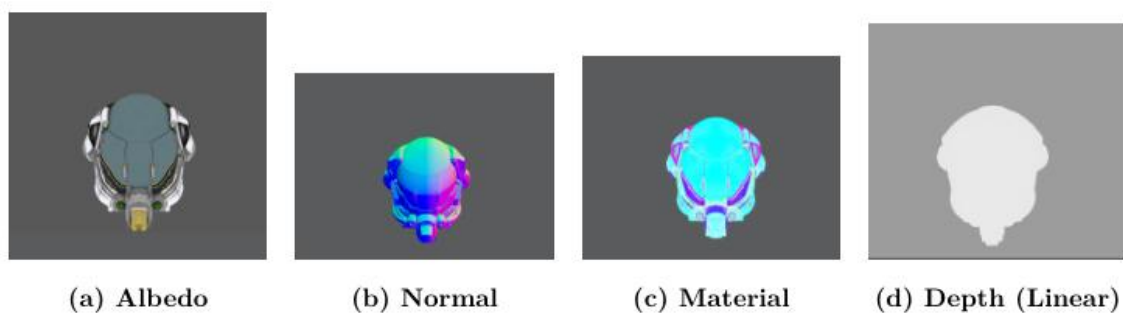


图 6.2 G-Buffer 多附件几何信号采集分量图

6.2.3 光线追踪与降噪算法对比

这如图 6.3 所示, 1-SPP 硬件光追打出来的原始阴影, 布满了密密麻麻的蒙特卡洛黑白噪点, 存在严重的视觉瑕疵。但经过管线后端的时空滤波处理后, 高频噪声被有效抹平, 得到了平滑的物理软阴影。



(a) 原始 1 SPP 间接光照 (高方差噪声)



(b) SVGF 滤波后结果 (平滑且保留细节)

图 6.3 SVGF 降噪前后的对比分析图

6.2.4 后期处理与最终画面输出

图 6.4 展示了是否开启 TAA 的效果对比。可以看到开启锯齿平滑, 提供画面精度。



(a) TAA OFF



(b) TAA ON

图 6.4 时域抗锯齿 (TAA) 开启前后视觉效果对比图

6.3 系统测试

渲染引擎不仅画质要达标, 底层的显存调度更是绝对不能出岔子。为了验证这个从零搭建的系统是不是个“一碰就崩溃”的半成品, 本节设计了几组针对动态数据流转和管线热切换的测试用例。

在测试全程, 保持开启了 Vulkan 官方的验证层 (Validation Layers)。一旦发生内存泄漏、管线死锁或是同步屏障设置错误, 控制台就会立刻飘红报错。表 6.1 记录了各项核心压力测试的执行步骤与实际通过情况。

表 6.1 系统核心功能测试用例表

测试 编号	测 试 功能	操作步骤	预期结果	测 试结果
T01	资 产 异步加载	从 UI 加载 200MB 的 GLTF 染 场景	界面无卡顿，模型完整渲 染	通 过
T02	渲 染 路径切换	在控制面板 中连续切换 Forward 与 Hybrid 渲染模式	RenderGraph 成功执行 DAG 拓扑重构，显存屏障合法， 画面正确更新且无崩溃	通 过
T03	AO 独立控制	勾选 Ray Traced AO 调节滑 块	画面即时产生柔和遮蔽感	通 过
T04	动 态 实体移除	选中模型并 点击 Remove	Deletion Queue 触发，系统 不崩溃	通 过
T05	调 试 预览输出	切换显示模 式至 Normal 分量 图	屏幕输出法线编码颜色视 图	通 过

6.4 本章小结

本章首先交代了跑通这套系统所需的软硬件底座。随后，拿出了真实的程序运行截图，逐一证实了 G-Buffer 剥离、光线追踪采样、SVGF 降噪这些写在纸面上的算法，都已经在引擎里转化为了实实在在的视觉效果。最后，借助几组颇具针对性的压力测试用例，证明了底层的 RenderGraph 调度和异步显存回收机制经受住了考验，有效规避了内存泄漏与设备宕机的风险。

第七章 总结与展望

7.1 全文工作总结

本文针对现代实时渲染中光线追踪计算开销大、低采样率下噪声泛滥的业界痛点，基于 Vulkan 1.3 图形 API 从零构建了一套高性能的实时混合渲染引擎。本文的核心工作与主要贡献总结如下：

1. 设计了基于无绑定架构与 RenderGraph 的自动化调度中枢：系统在资源层实现了全局无绑定架构与基于 RAI 的显存生命周期托管，消除了传统的逐对象绑定开销。在调度层，设计并实现了数据驱动的 RenderGraph 架构，利用 DAG 实现了渲染任务的逻辑解耦；通过自动化推导资源状态生成精确的同步屏障（Barrier），彻底解决了 Vulkan 异构计算中手动同步繁琐且易错的问题，并实现了时域历史资源的持久化，为后续算法奠定了底层框架。

2. 构建了结合光源重要性采样的硬件混合渲染管线：系统充分结合了光栅化 G-Buffer 特征提取的高效性与硬件光线追踪（RT Pipeline）的物理精确性。针对单次采样（1-SPP）极低采样率导致的方差过高的问题，系统在 CPU 端预先评估了场景自发光体的物理辐射度，构建了光源能量的累计分布函数（CDF）。在光线探测阶段，系统引入了基于光源能量分布的重要性采样（Importance Sampling），有效引导射线向高贡献度光源发射，极大地抑制了原始物理探测信号的初始噪点，显著提升了场景软阴影与全局光影的收敛速度。

3. 落地了高性能的时空信号重建与后期处理系统：深入研究并实现了 SVGF 算法，在极低采样率下通过历史重投影累积与自适应的联合双边 A-Trous 空间滤波，实现了高质量的光影降噪。此外，系统集成了时域抗锯齿（TAA）与 ACES 色调映射后期处理方案，通过亚像素抖动与色彩空间方差裁剪技术，消除了几何边缘走样并还原了极具质感的 PBR 物理画质。

7.2 研究不足与未来展望

受限于有限的个人开发周期与单机显卡的算力瓶颈，本系统在现阶段主要解决了混合渲染架构“从无到有”的底层流转问题。若要在未来进一步拔高引擎的画面表现力与运行帧率，后续的迭代工作将重点围绕以下四个技术痛点展开：

目前的管线主要精力被倾注在直接光软阴影与单次弹射的间接光照上。想要在有限的算力预算内,实现逼真的多弹射全局光照(Multi-bounce GI)甚至复杂的焦散(Caustics)效果,下一步可以引入 ReSTIR(储层时空重要性重采样)算法。通过在时间与空间域上复用有效光线样本,提高实时光线追踪的效率。

引擎目前的 PBR 材质库仅完整跑通了基础的 Cook-Torrance 模型。但底层的着色器与资源管理架构已经预留了充足的扩展空间。未来的开发可以直接在此基础上,引入次表面散射(SSS,用于渲染真实的皮肤与玉石透射)、清漆层(Clearcoat)以及各向异性(Anisotropy)等高级材质特性,全面提升引擎对复杂物理世界的模拟能力。

虽然本项目手写的 SVGF 配合 TAA 勉强压制住了 1-SPP 带来的满屏噪点,但一旦场景出现剧烈的动态变化,依然难以避免重投影失败导致的鬼影(Ghosting)和画面糊化。借助于当前 RTX 5070 Ti 等现代显卡内置的 Tensor Core 算力,未来完全可以抛弃传统的启发式滤波,直接集成 NVIDIA DLSS 3.5(Ray Reconstruction)或 AMD FSR 等前沿 SDK,利用深度学习模型降维打击复杂的信号重建难题。

在渲染性能的深度挖掘上,当下的 RenderGraph 调度系统虽然理清了依赖,但尚未将显存物理复用(Aliasing)的极限压榨出来。后续可引入基于资源生存区间的绝对重叠分配策略。更重要的是,基于 Vulkan 1.3 规范,未来可以全面接入网格着色器(Mesh Shaders),将原本由 CPU 负责的八叉树遍历与视锥体剔除工作全盘下放到计算着色器中执行,向纯 GPU 驱动渲染的架构靠拢。

参考文献

- [1] AKENINE-MOLLER T, HAINES E, HOFFMAN N. Real-time rendering [M]. Wellesley, MA: AK Peters/crc Press, 2019.
- [2] LIU E. Low Sample Count Ray Tracing with NVIDIA's Ray Tracing Denoisers; proceedings of the SIGGRAPH 2018 NVIDIA Exhibitor Session, Vancouver, F, 2018 [C]. NVIDIA.
- [3] ANDERSSON J, BARRE-BRISEBOIS C. DirectX: Evolving Microsoft's Graphics Platform [Z]. Microsoft Sponsored Session, Game Developers Conference. 2018.
- [4] STACHOWIAK T. Stochastic all the things: Raytracing in hybrid real-time rendering [J]. Digital Dragons, 2018: 21-2.
- [5] SANGLARD F. Game Engine Black Book: Wolfenstein 3D [M]. [S.l.]: CreateSpace Independent Publishing Platform, 2017.
- [6] SANGLARD F. Game Engine Black Book: Doom [M]. [S.l.]: CreateSpace Independent Publishing Platform, 2018.
- [7] O'DONNELL Y. FrameGraph: Extensible rendering architecture in Frostbite; proceedings of the Game Developers Conference 2017, San Francisco, F, 2017 [C]. UBM.
- [8] SCHIED C, KAPLANYAN A, WYMAN C, et al. Spatiotemporal variance-guided filtering [Z]. Proceedings of High Performance Graphics. 2017: 1-12.10.1145/3105762.3105770.
- [9] HAINES E, AKENINE-MÖLLER T. Ray Tracing Gems: High-Quality and Real-Time Rendering with DXR and Other APIs [M]. Berkeley, CA: Apress, 2019.
- [10] WHITTET T. An improved illumination model for shaded display [M]. ACM Siggraph 2005 Courses. 2005: 4-es.
- [11] BURLEY B, STUDIOS W D A. Physically-based shading at disney; proceedings of the Acm siggraph, F, 2012 [C]. vol. 2012.
- [12] WALTER B, MARSCHNER S R, LI H, et al. Microfacet models for refraction through rough surfaces [Z]. Proceedings of the 18th Eurographics conference on Rendering Techniques. Grenoble, France; Eurographics Association. 2007: 195–206.
- [13] RENDERING W P-B. Physically-based rendering [J]. Procedia IUTAM, 2015, 13(127-137): 3.
- [14] COOK R L, TORRANCE K E. A reflectance model for computer graphics [J]. ACM SIGGRAPH Computer Graphics, 1981, 15(3): 307-16.
- [15] TROWBRIDGE T, REITZ K P. Average irregularity representation of a rough surface for ray reflection [J]. Journal of the optical society of America, 1975, 65(5): 531-6.
- [16] SELLERS G, KESSENICH J. Vulkan programming guide: The official guide to learning vulkan [M]. Boston, MA: Addison-Wesley Professional, 2016.
- [17] MARRS A, SHIRLEY P, WALD I. Ray tracing Gems II: next generation real-time rendering with DXR, Vulkan, and OptiX [M]. Berkeley, CA: Springer Nature, 2021.
- [18] GPUOPEN-LIBRARIESANDSDKS. Vulkan Memory Allocator [Z]. GitHub. 2017.
- [19] CORNUT O. Dear ImGui: Bloat-free graphical user interface for C++ with minimal dependencies [Z]. GitHub. 2014.
- [20] CHERNIKOV Y. Walnut: A lightweight C++ application framework [Z]. GitHub. 2022.
- [21] GREGORY J. Game engine architecture [M]. Boca Raton, FL: AK Peters/CRC Press, 2018.

- [22] XU K. Temporal antialiasing in Uncharted 4 [J]. ACM SIGGRAPH Courses: Advances in Real-Time Rendering in Games, 2016, 3(9): 10.
- [23] KAPOULKINE A. volk: Meta loader for Vulkan API [Z]. GitHub. 2020
- [24] PETTINEO M. Bindless texturing for deferred rendering and decals [Z]. The Danger Zone blog. 2016: 192-907
- [25] SALVI M. An excursion in temporal supersampling; proceedings of the Game Developers Conference 2016, San Francisco, F, 2016 [C]. UBM.

致谢

时光荏苒，本科学习生涯即将落幕，毕业设计的完成也为这段求学之旅画上圆满句号。从论文选题、框架搭建到最终定稿，每一步都离不开各位老师、家人与朋友的帮助，在此，我谨向所有给予我支持的人致以最诚挚的谢意。

首先衷心感谢我的指导老师韩强老师，老师以严谨的治学态度和耐心的指导为本文的撰写提供了诸多帮助。然后感谢学院各位授课老师，本科期间传授的专业知识与思维方法，为本次毕业设计奠定了坚实基础，也让我提升了专业素养与独立思考能力。也感谢我的家人，他们是最坚实的后盾，始终给予我无私的关爱与支持，为我创造安心的学习环境，他们的期盼是我前进的不竭动力。

最后，再次感谢所有在论文撰写中给予我帮助的人们。